

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce and Science College, Bambhori, Jalgaon
Bachelor of Computer Application (B.C.A.)**



Bachelor of Computer Applications

**LAB MANUAL ON
606 Android Application Development**

Name:

Class:

Sem:

Roll No:

Seat No:

Name of faculty: Ms.Swati.S.Badhe

Academic Year: 2025 - 2026

Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)

Practical: 01

DOP:

DOC:

Title: Installation and setup of java development kit (JDK), setup android SDK, setup eclipse IDE, setup android development tools (ADT) plugins, create android virtual device.

Objective: To study and perform the installation and proper configuration of Java Development Kit (JDK), Android Software Development Kit (SDK), Eclipse IDE with Android Development Tools (ADT) plugin, and to create an Android Virtual Device (AVD) for developing and testing Android applications.

Requirements:

1. Computer system with Windows operating system
2. Java Development Kit (JDK)
3. Android Software Development Kit (SDK)
4. Eclipse IDE for Java Developers
5. Android Development Tools (ADT) plugin
6. Internet connection for downloading required software
7. Minimum 4 GB RAM and sufficient disk space

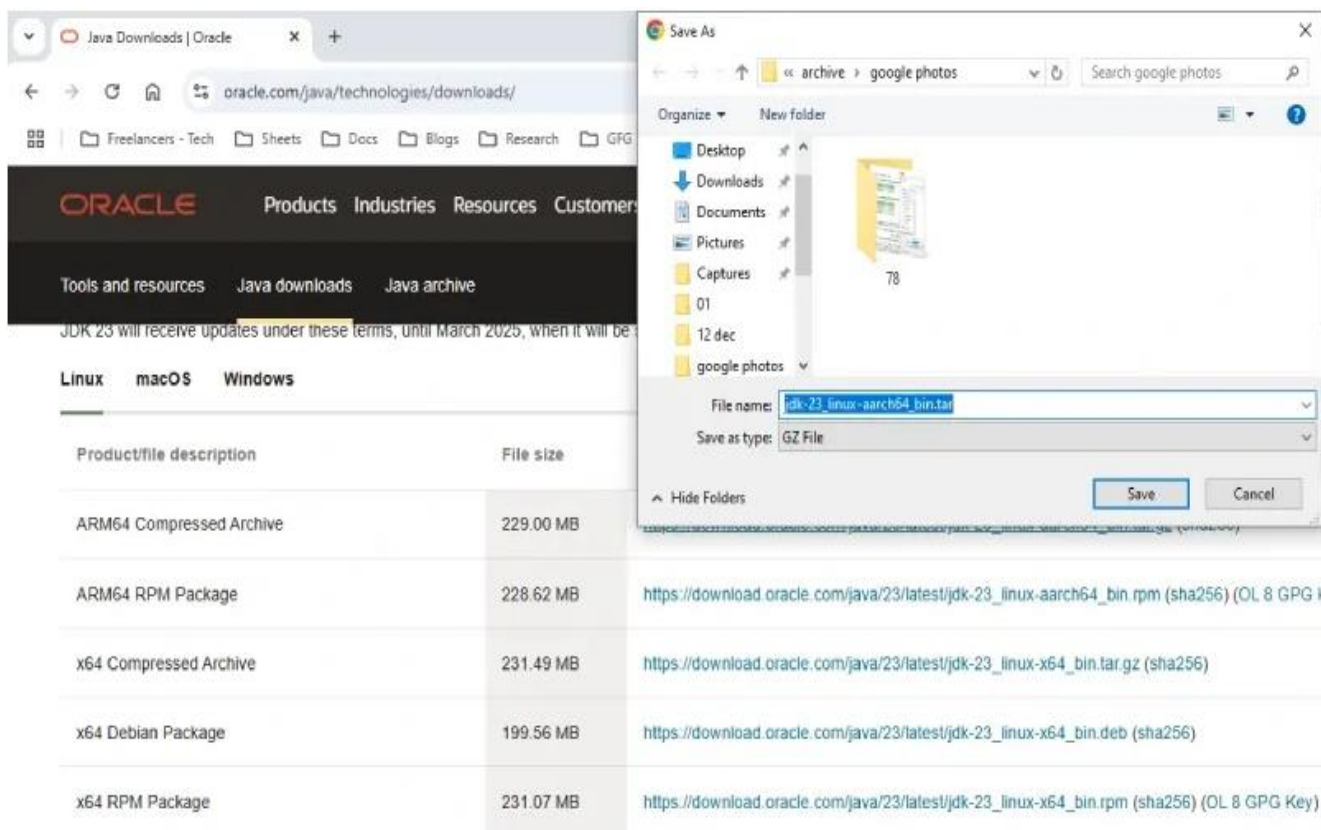
Step-by-Step Instructions:

Installation and Setup of Android Development Environment:

Step 1: Installation of Java Development Kit (JDK)

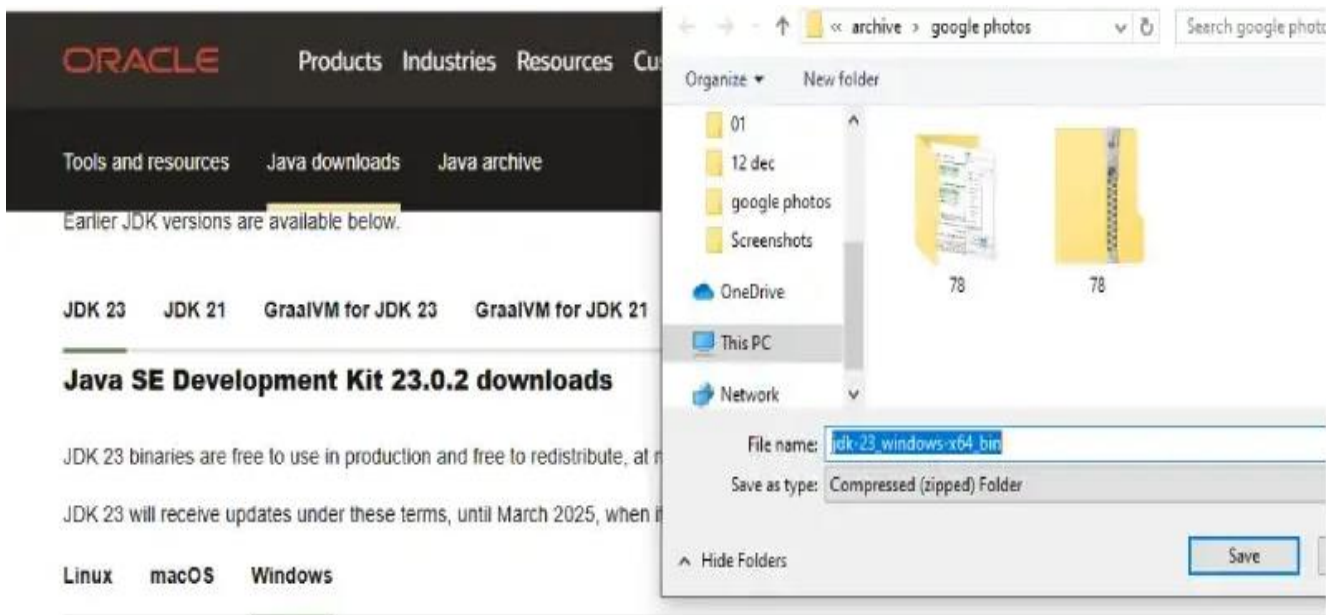
1. Open a web browser and download the latest JDK from the official Oracle website.

Go to the <https://www.oracle.com/java/technologies/downloads/> to download the file.



Step 2: Select the Appropriate Version

As of 2025, the latest stable versions are JDK 23 (SE) and JDK 21 (LTS). Select the compatible version as per your operating system (Windows, Mac or Linux)



Step 3: License and Agreement

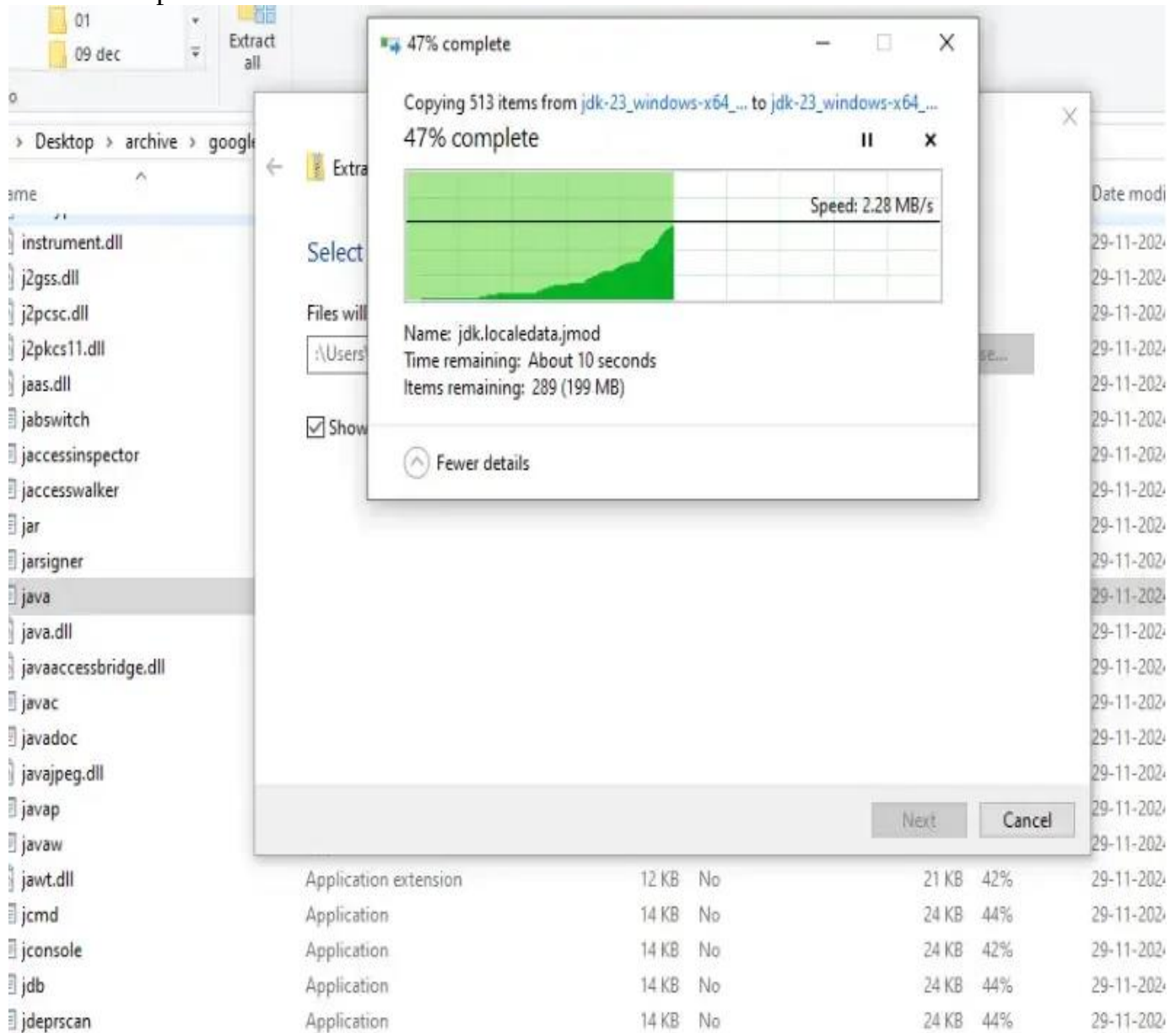
Go through all the License and Agreement before downloading (from Oracle website), it will not ask if you'll download it from OpenJDK website.

Install JDK on Windows

Follow the below steps to install JDK on Windows environment i.e. Windows 7, Windows 8, Windows 8.1, Windows 10 and Windows 11.

Step 1: Run the Java Development Kit (JDK) Installer

Locate the downloaded .exe file (e.g. jdk-23-windows-x64_bin.exe) and make the double click to begin the Installation process. Follow the installation wizard prompts to complete the installation process.

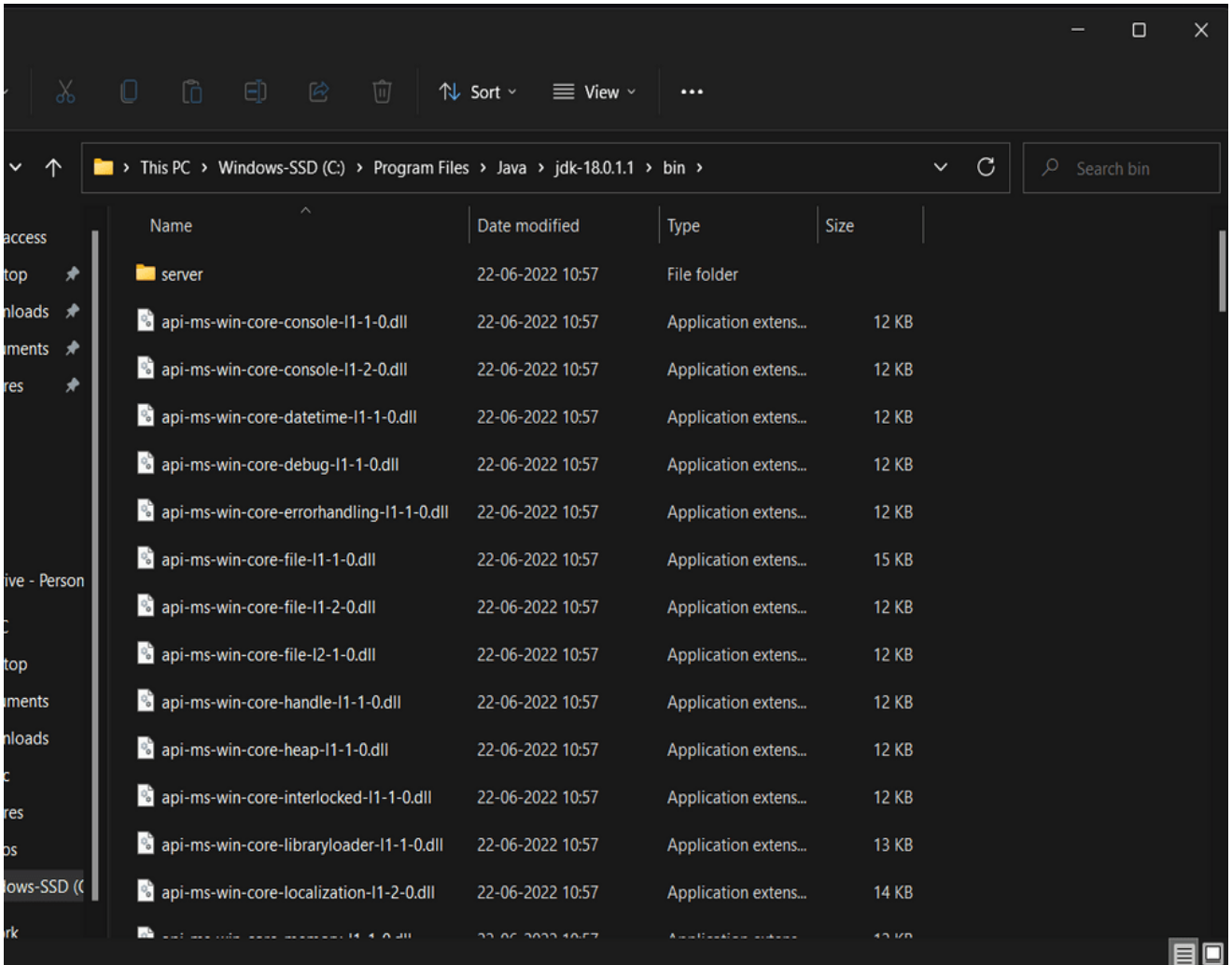


JDK Installer for Windows

Step 2: Setup the Environment Variables

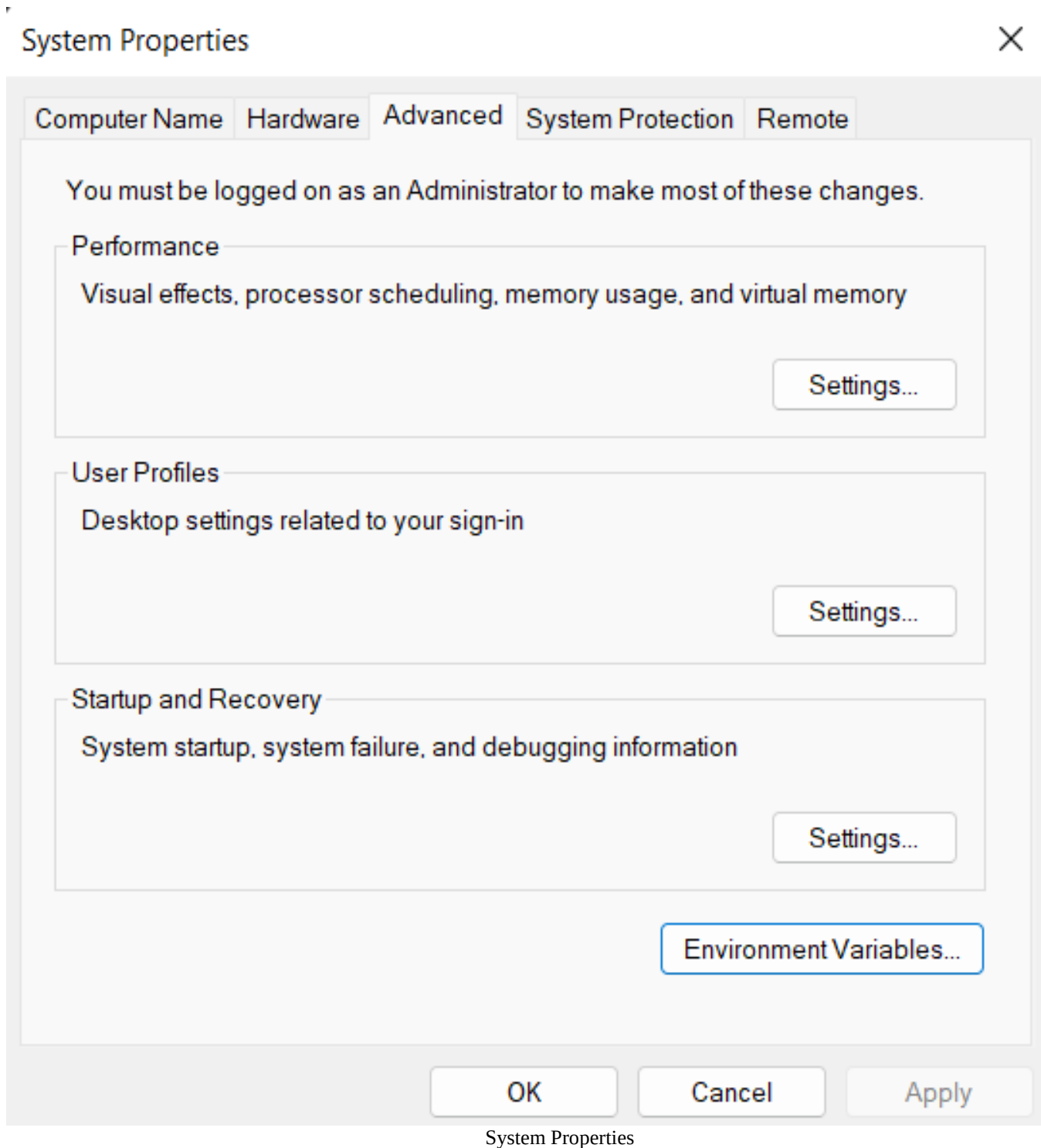
Once the installation gets completed, you need to configure environment variables to notify the system about the directory in which the JDK files are located.

Proceed to **C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}\bin** (replace {-} with your JDK version)

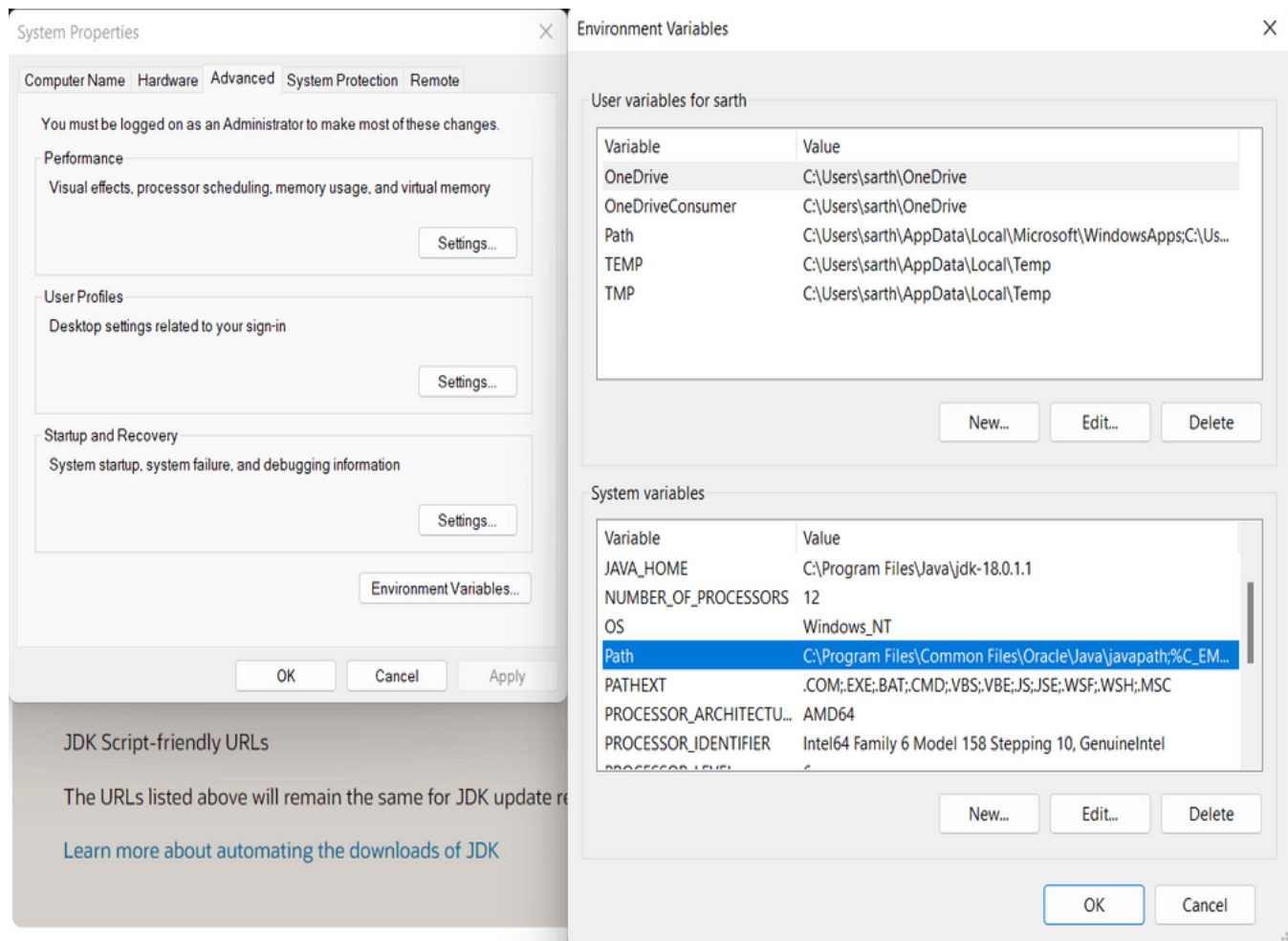


Program Files

Step 2.1: To set the Environment Variables, you need to search Environment Variables in the [Task Bar](#) and click on “Edit the system environment variables”.

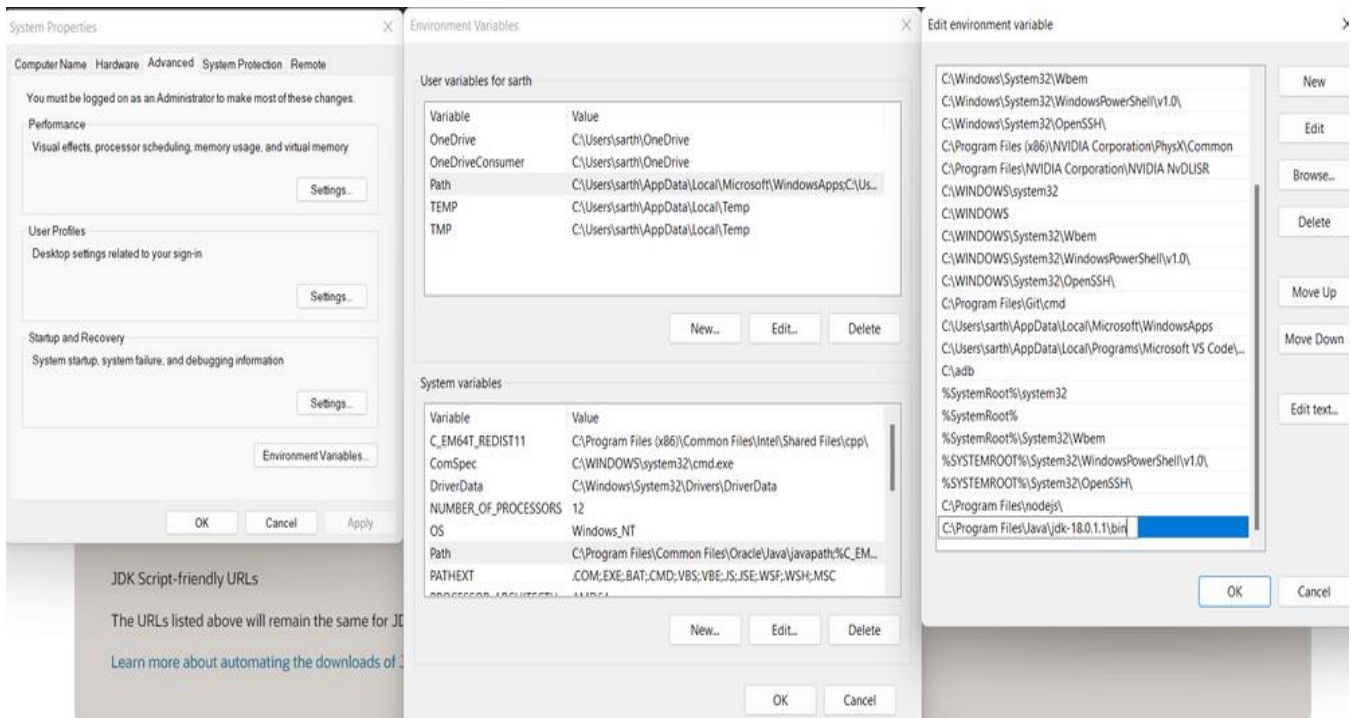


Step 2.2: Under the Advanced section, Click on "Environment Variables".



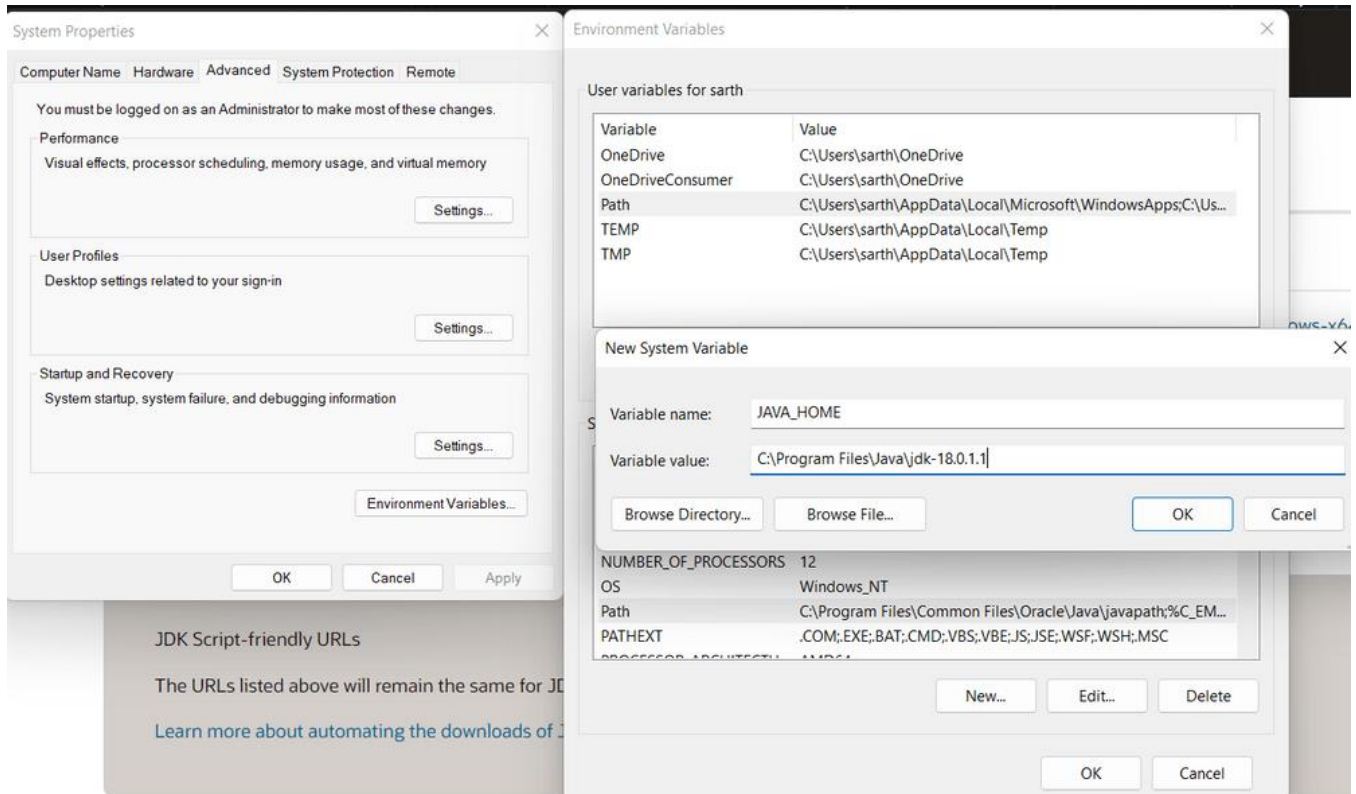
Environment Variables

Step 2.3: Under System variables, select the "Path" variable and click on "Edit". Click on "New" then paste the Path Address i.e. C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}\bin. Click on "OK".



Path

Step 2.4: Now, in the Environment Variables dialogue, under System variables, click on "New" and then under Variable name: JAVA_HOME and Variable value: paste address i.e. *C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}*. Click on OK => OK => OK



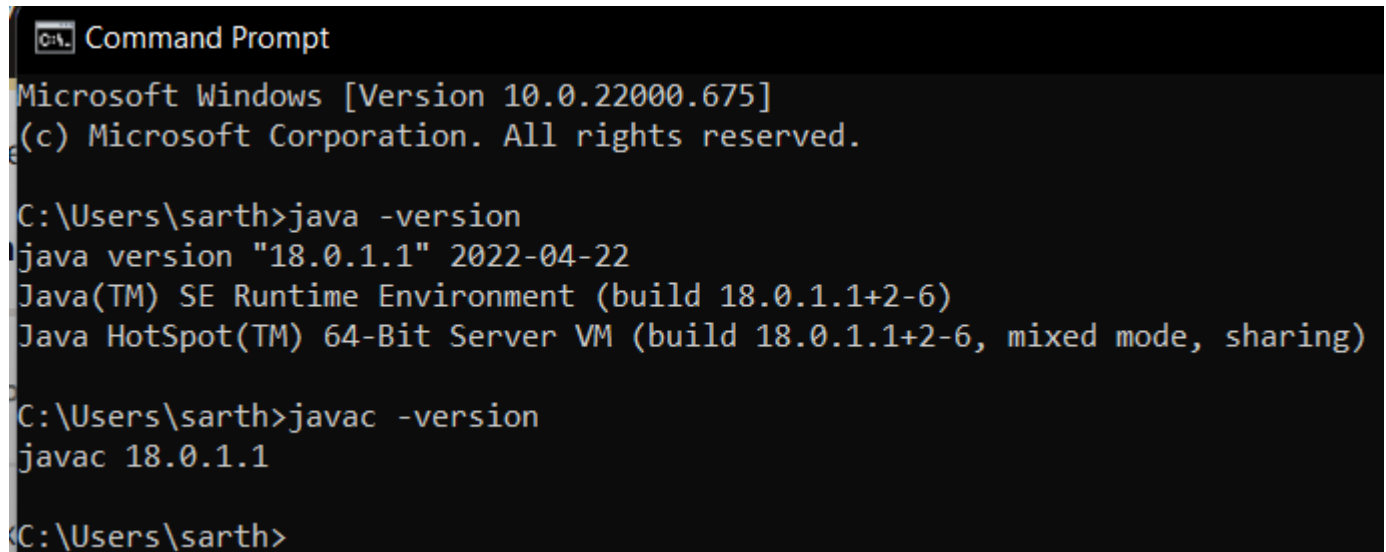
System Properties

Step 3: Check the Java Version

Open [Command Prompt](#) and enter the following commands:

```
java -version
```

```
javac -version
```

A screenshot of a Windows Command Prompt window. The title bar says 'Command Prompt'. The text inside shows the Windows version (10.0.22000.675) and copyright information. The user 'sarth' has entered 'java -version' and 'javac -version'. The output for 'java -version' shows '18.0.1.1' and '2022-04-22'. The output for 'javac -version' shows '18.0.1.1'.

```
Microsoft Windows [Version 10.0.22000.675]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sarth>java -version
java version "18.0.1.1" 2022-04-22
Java(TM) SE Runtime Environment (build 18.0.1.1+2-6)
Java HotSpot(TM) 64-Bit Server VM (build 18.0.1.1+2-6, mixed mode, sharing)

C:\Users\sarth>javac -version
javac 18.0.1.1

C:\Users\sarth>
```

Command Prompt

2. Steps to Install Android Studio on Windows

Step 1: Head over to [this link](#) to get the Android Studio executable or zip file.

Step 2: Click on the **Download Android Studio** Button.

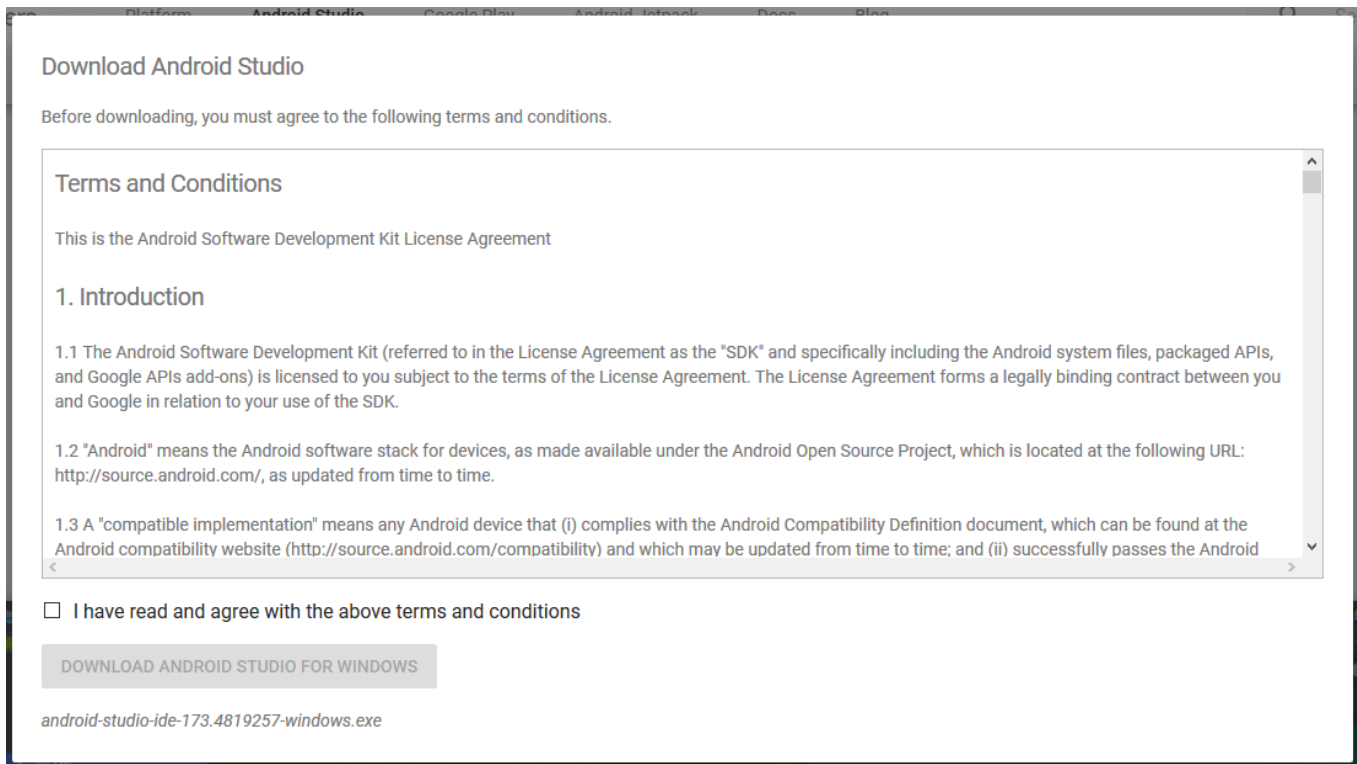


Android Studio provides the fastest tools for building apps on every type of Android device.

DOWNLOAD ANDROID STUDIO

4.1.3 for Windows 64-bit (896 MiB)

Click on the "I have read and agree with the above terms and conditions" checkbox followed by the download button.



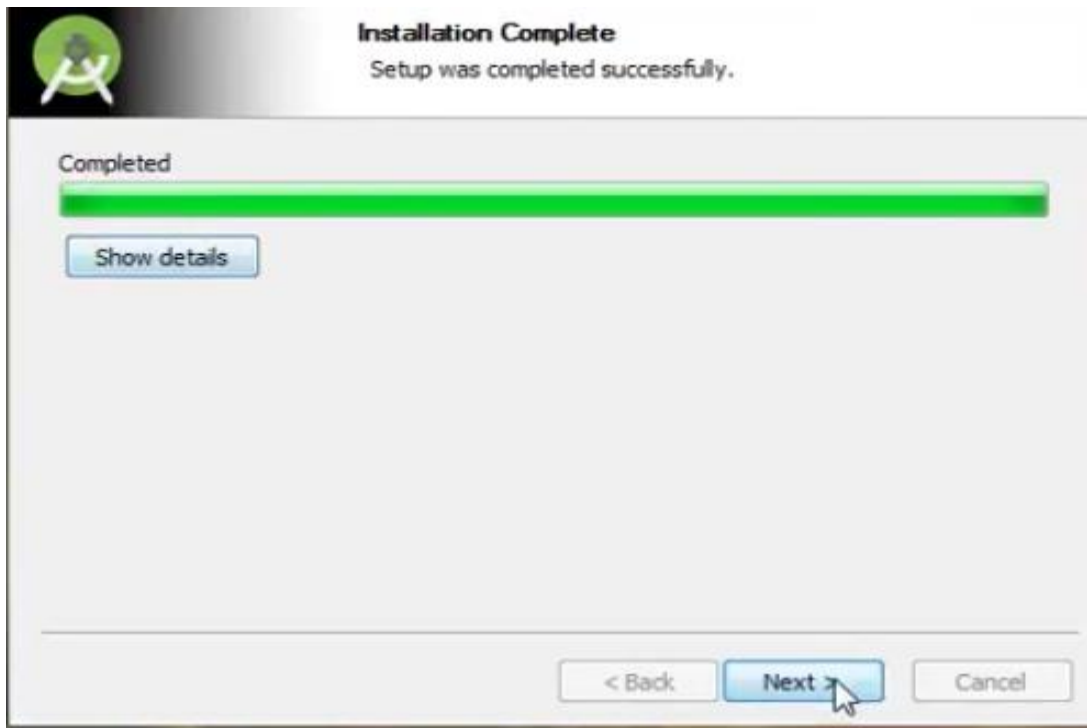
Click on the Save file button in the appeared prompt box and the file will start downloading.

Step 3: After the downloading has finished, open the file from downloads and run it. It will prompt the following dialog box.



Click on next. In the next prompt, it'll ask for a path for installation. Choose a path and hit next.

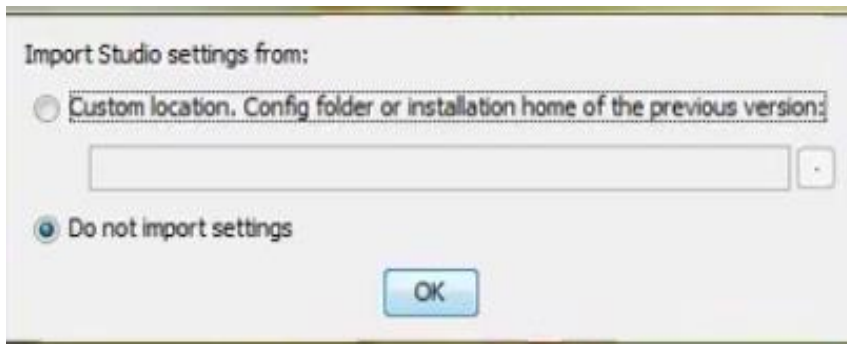
Step 4: It will start the installation, and once it is completed, it will be like the image shown below.



Click on next.



Step 5: Once " **Finish** " is clicked, it will ask whether the previous settings need to be imported [if the android studio had been installed earlier], or not. It is better to choose the 'Don't import Settings option'.

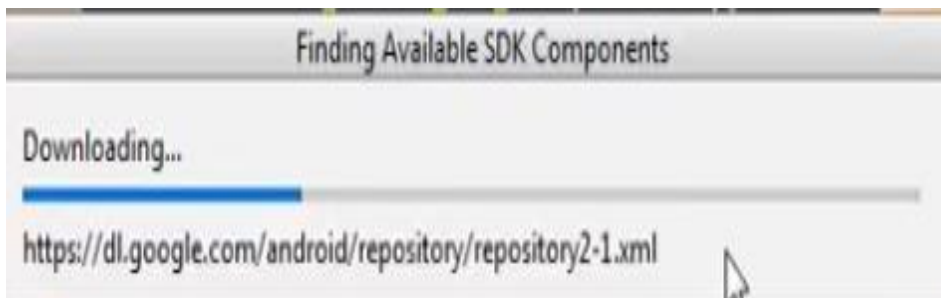


Click the **OK** button.

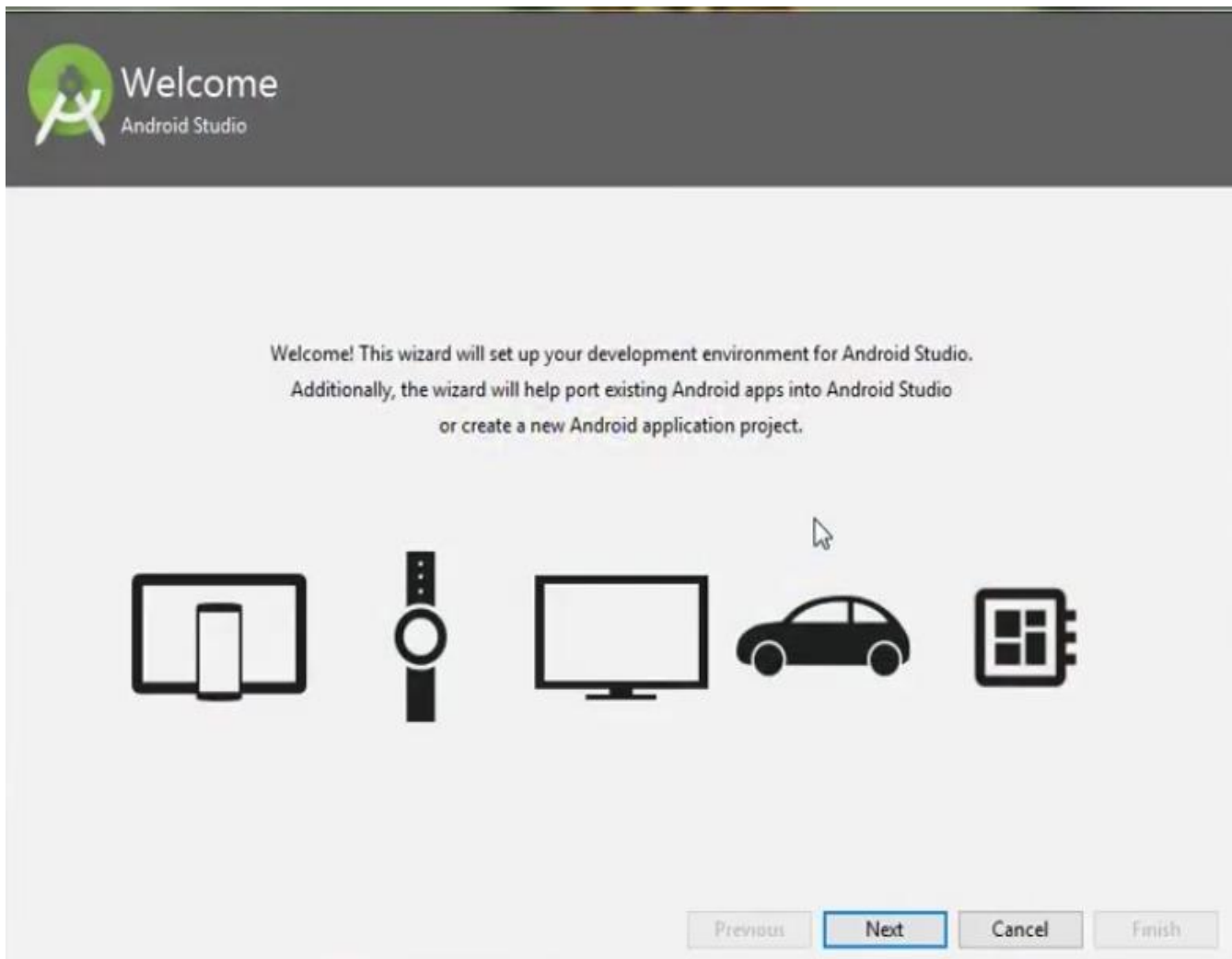
Step 6: This will start the Android Studio.



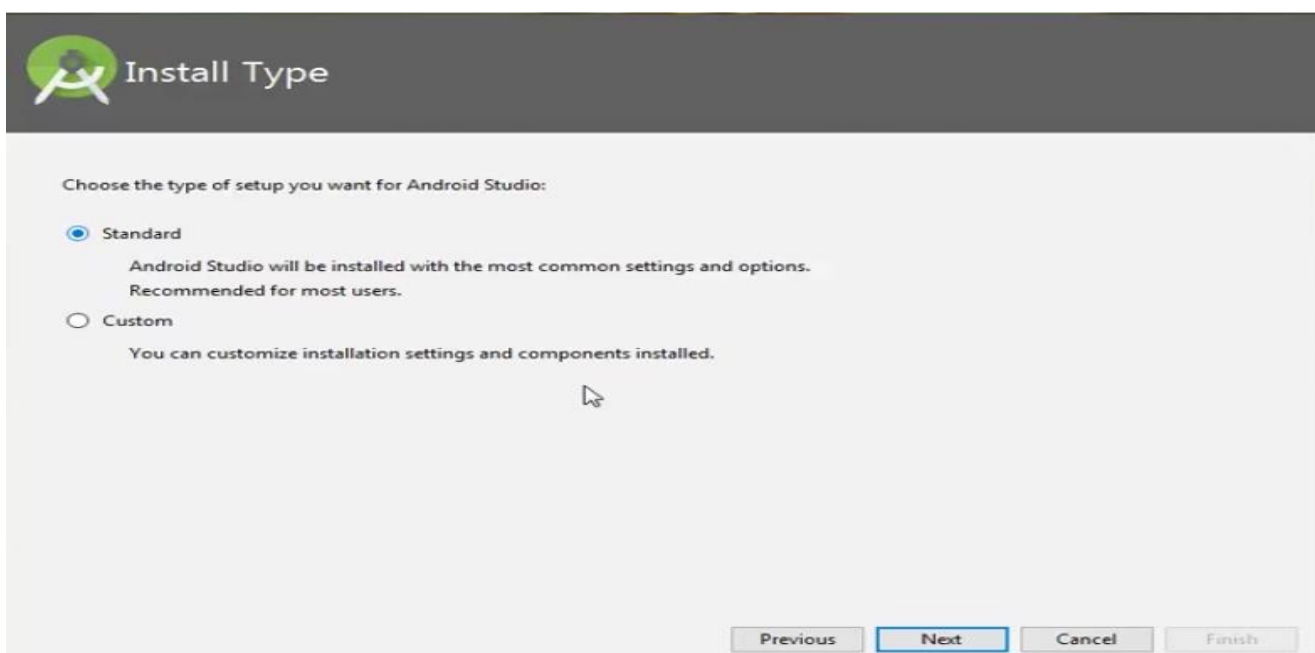
Meanwhile, it will be finding the available SDK components.



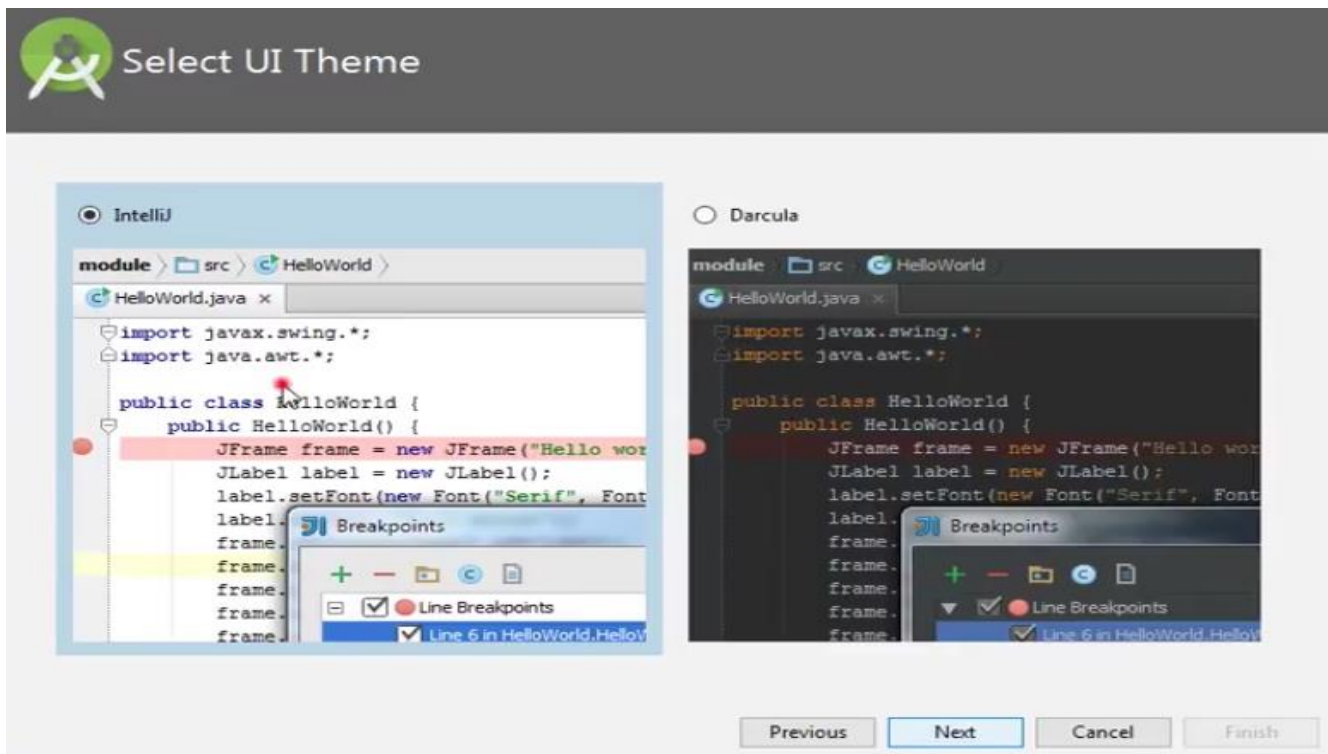
Step 7: After it has found the SDK components, it will redirect to the Welcome dialog box.



Click on **Next** .

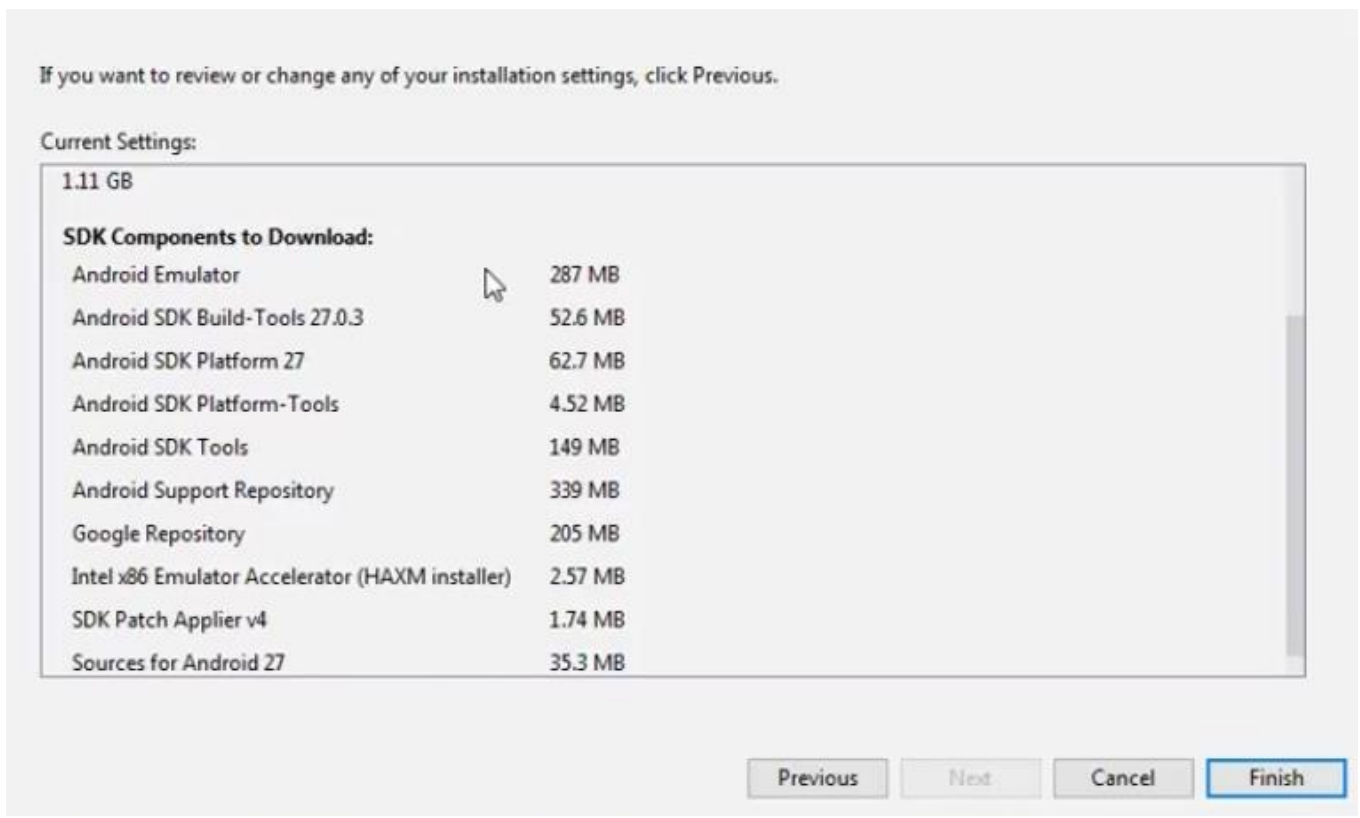


Choose Standard and click on Next. Now choose the theme, whether the **Light** theme or the **Dark** one. The light one is called the **IntelliJ** theme whereas the dark theme is called **Dracula** . Choose as required.

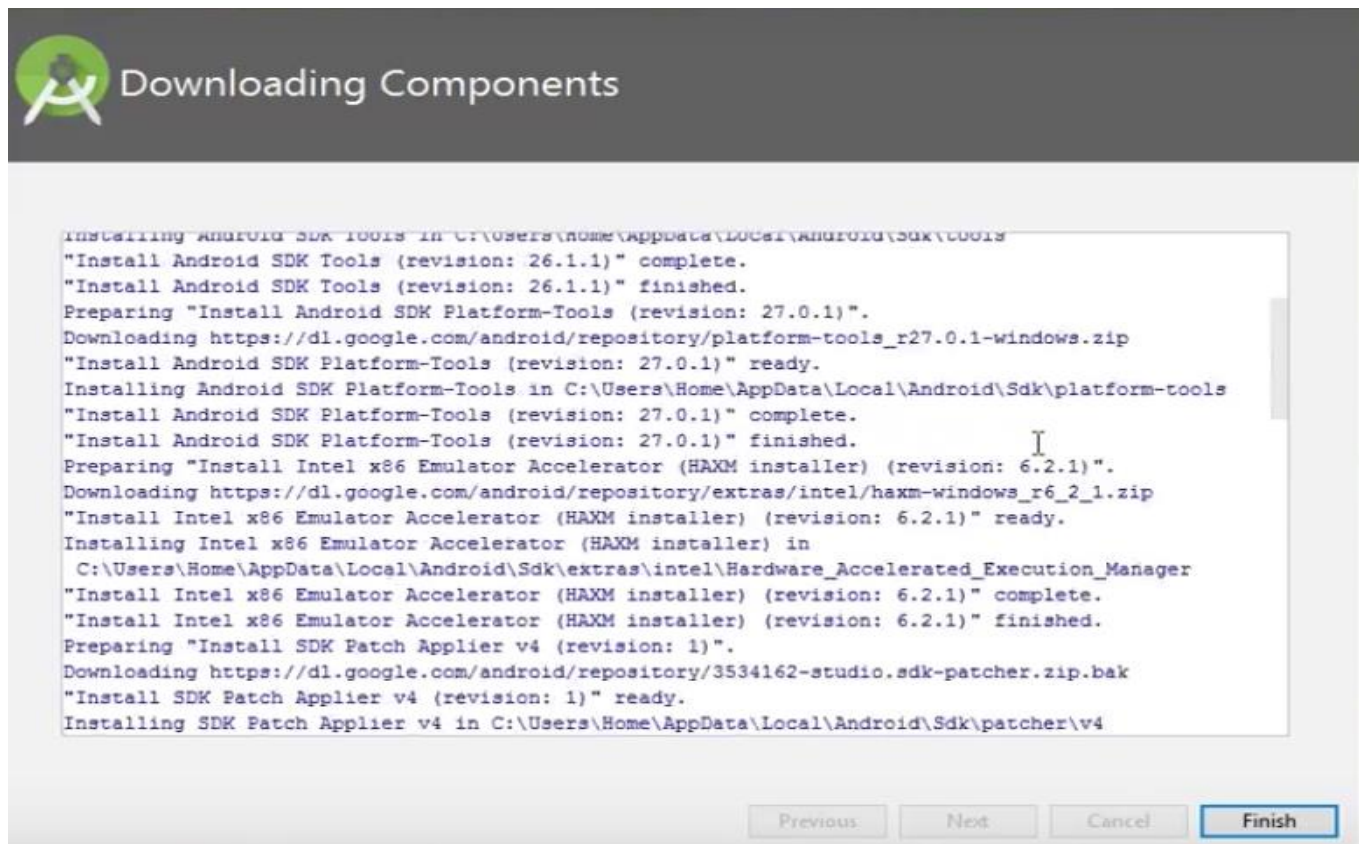


Click on the **Next** button.

Step 8: Now it is time to download the SDK components.



Click on Finish. Components begin to download let it complete.



The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

Step 9: Click on **Start a new Android Studio project** to build a new app.

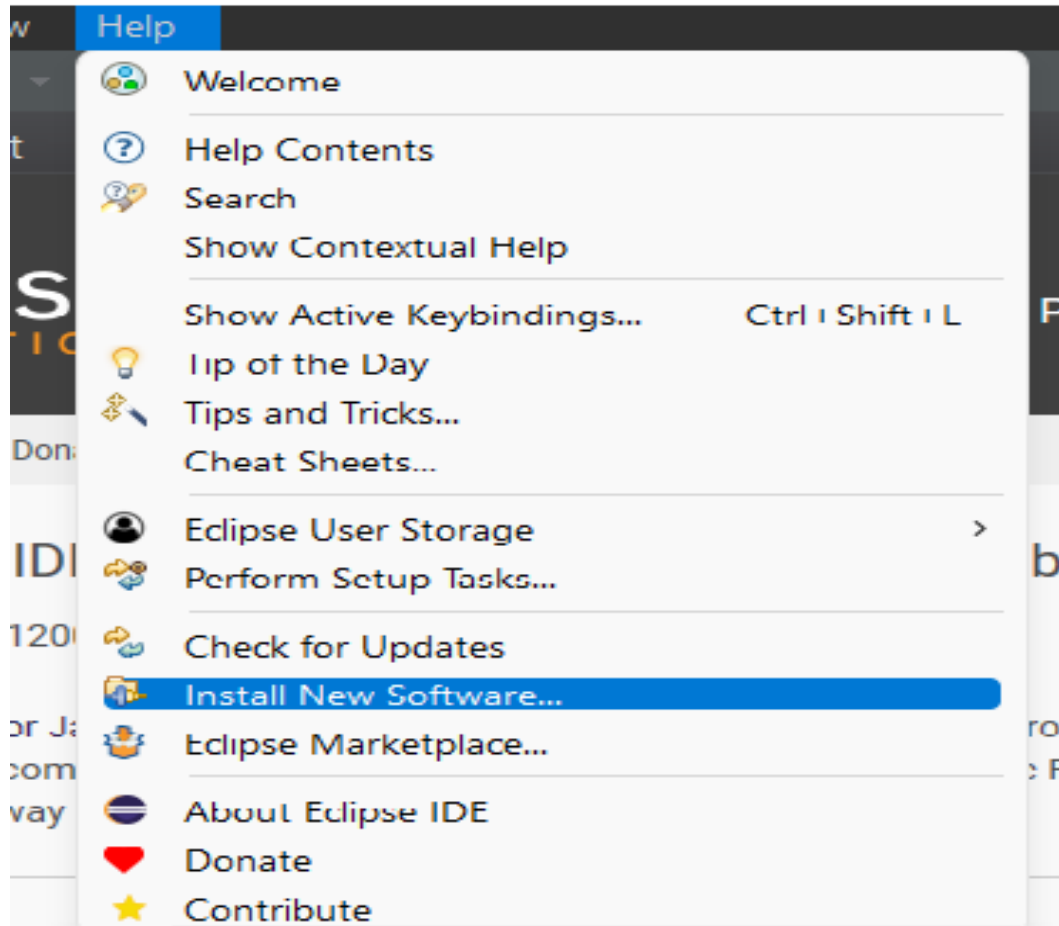


3. Steps to Setup Eclipse IDE and setup android development tools (ADT) plugins for Android App Development

Step 1: Open Eclipse IDE.

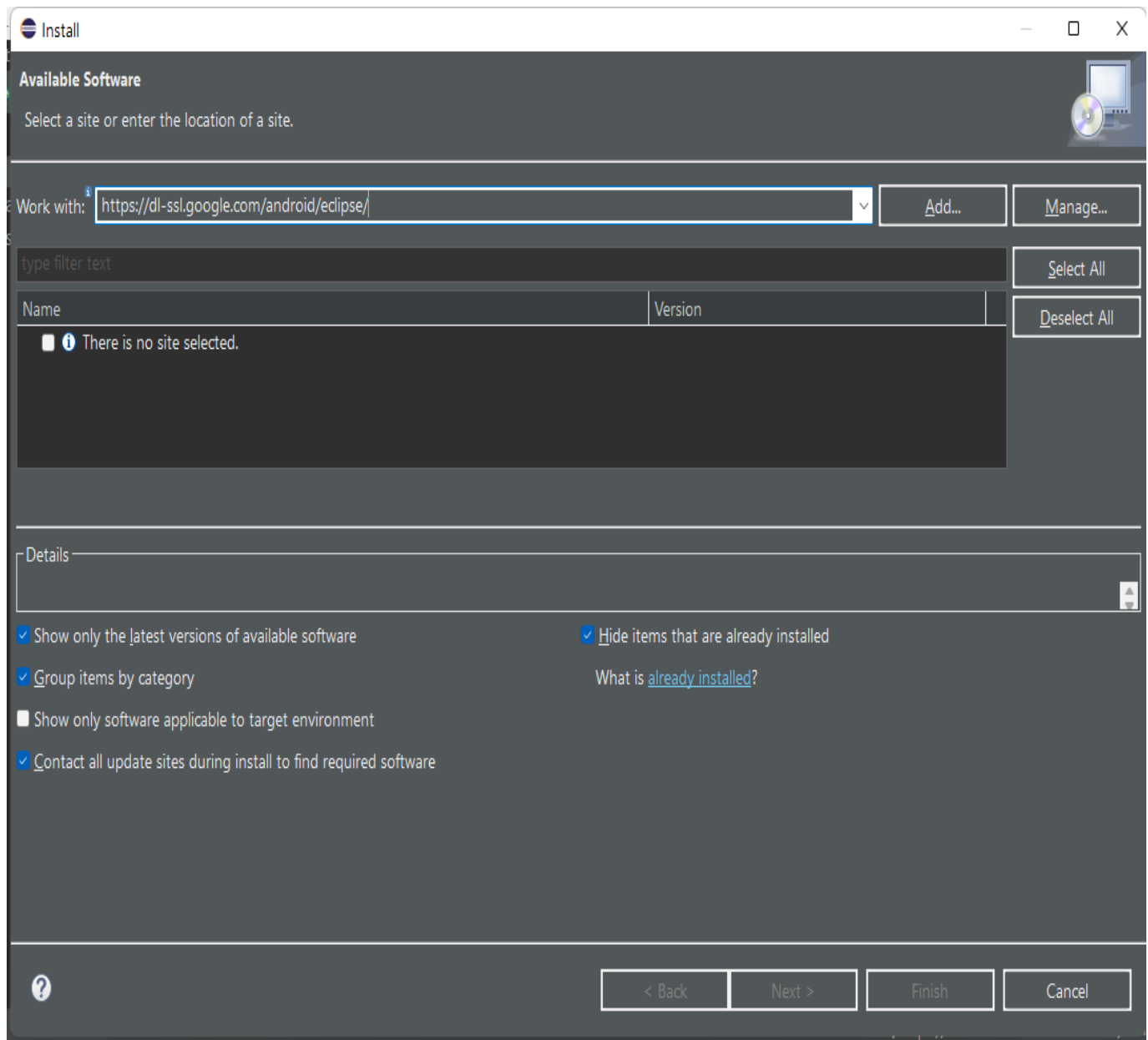
Step 2: Select Help, Click on "Install New Software"

'scope=Eclipse%20IDE%20for%20Java%20Developers%20(in

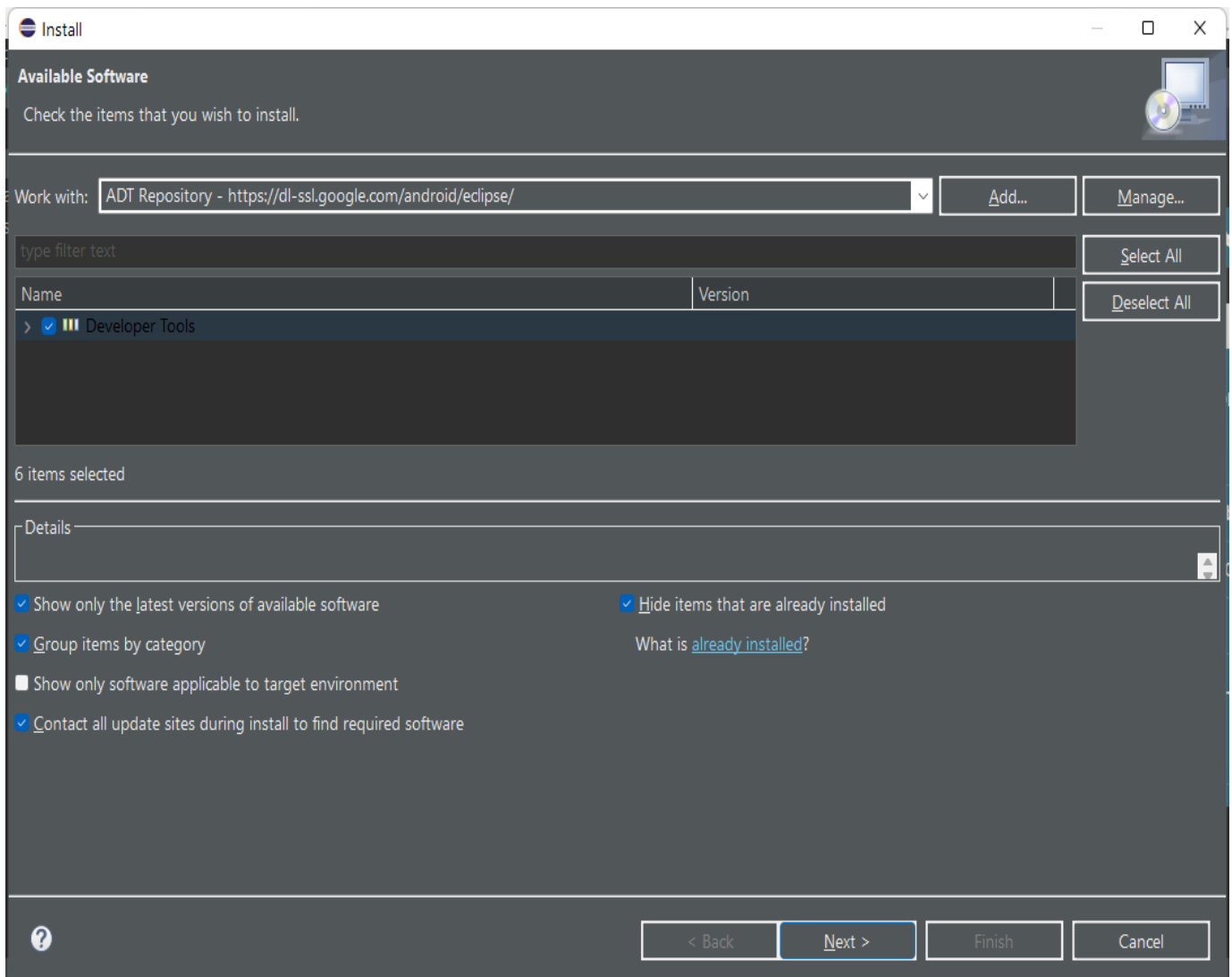


I Do to Support the Eclipse Commu

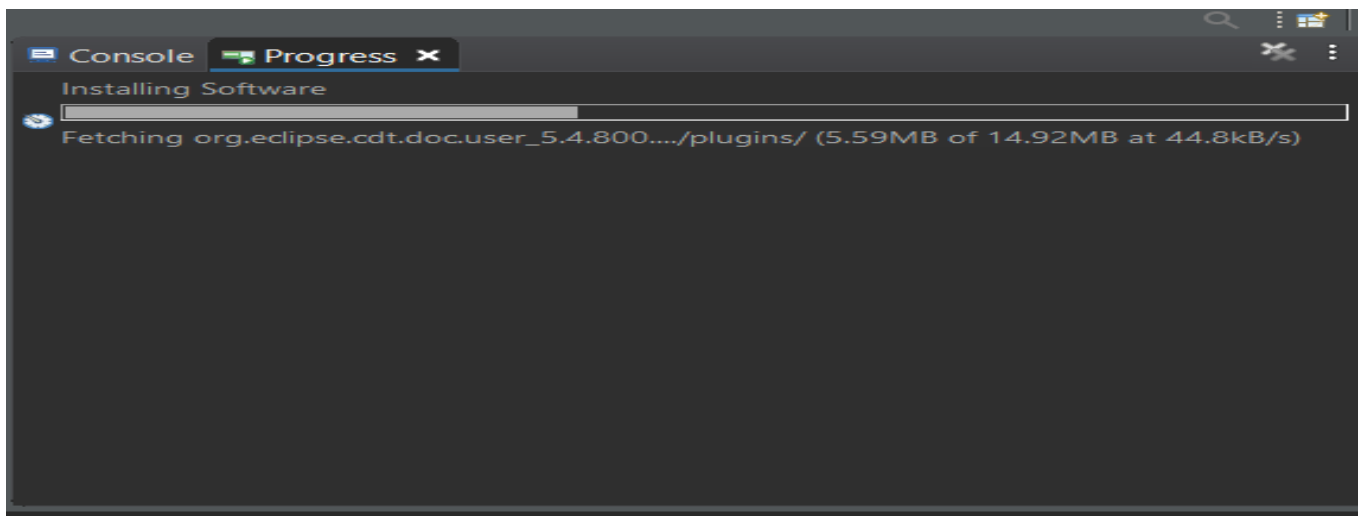
Step 3: Type "<https://dl-ssl.google.com/android/eclipse/>" in the "Work With" section and click on Add. Further, a new Dialog box will appear, type Name - ADT Repository and Click on Add.



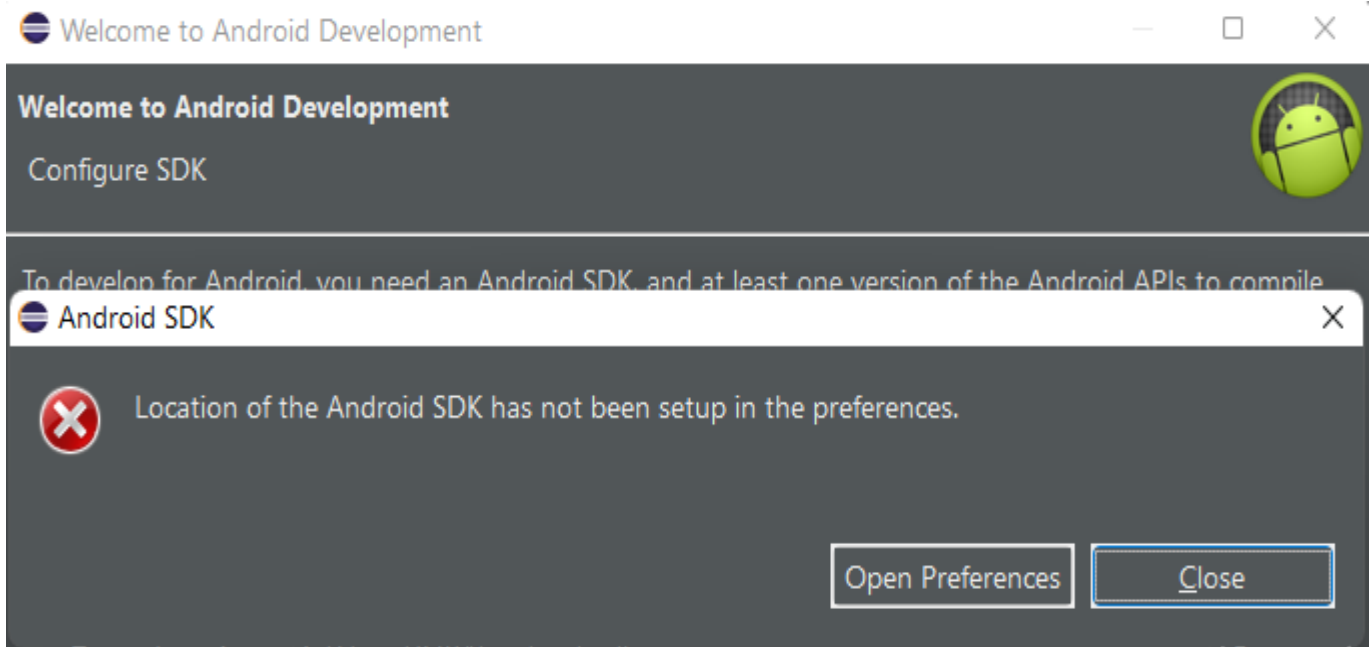
Step 4: You can see Developer Tools under Name - tick the box, then click Next. A dialog box will appear, click on Next and then click on Finish. After that, Installation will begin.



The installation will take some time:

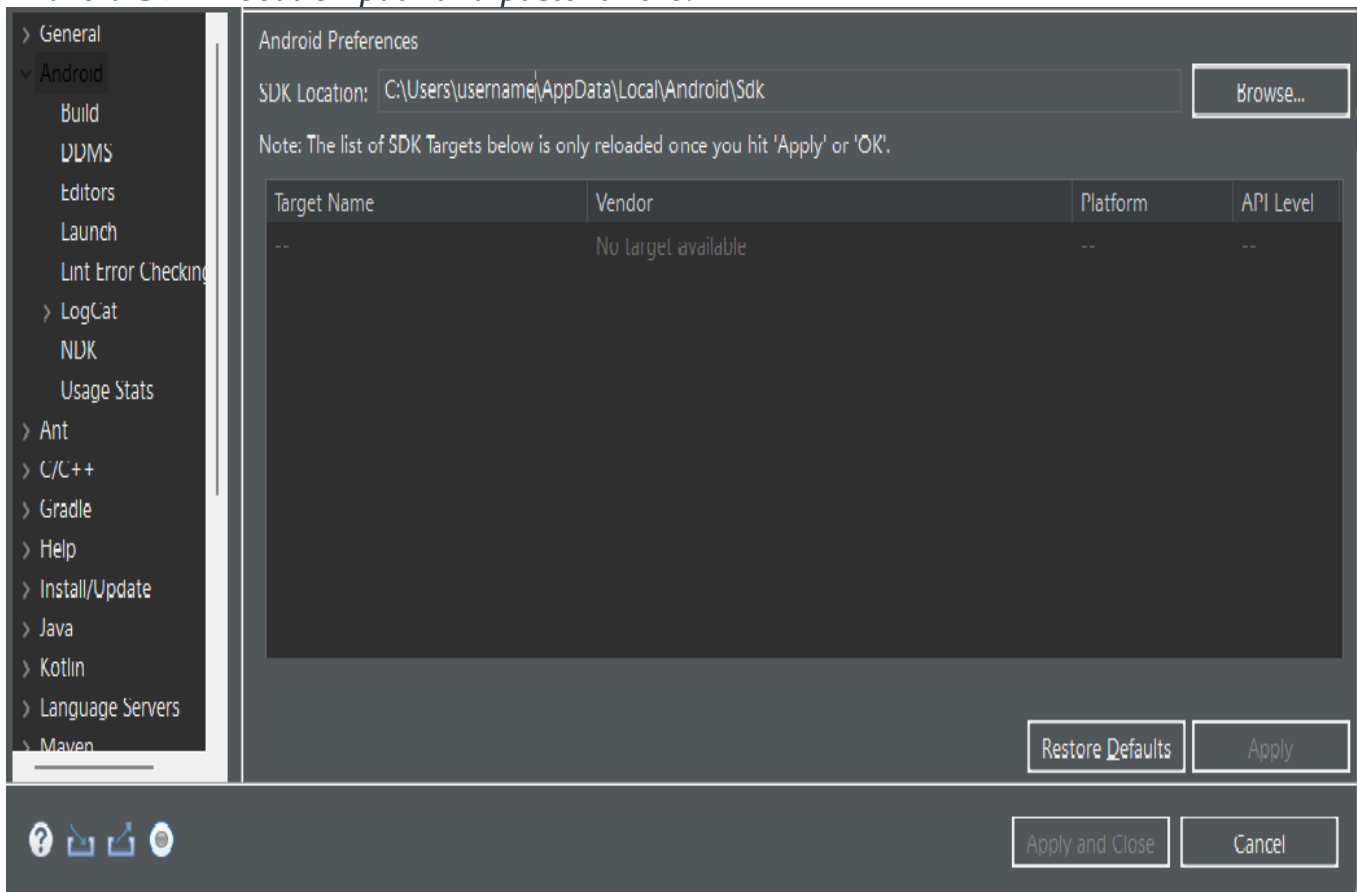


Step 5: After the installation is completed, Eclipse will be restarted. After the restart, a dialog box will appear for setting up the Preferences. Click on Open Preferences then Click on Proceed. If the dialog box does not appear then go to Eclipse -> Window -> Preferences.

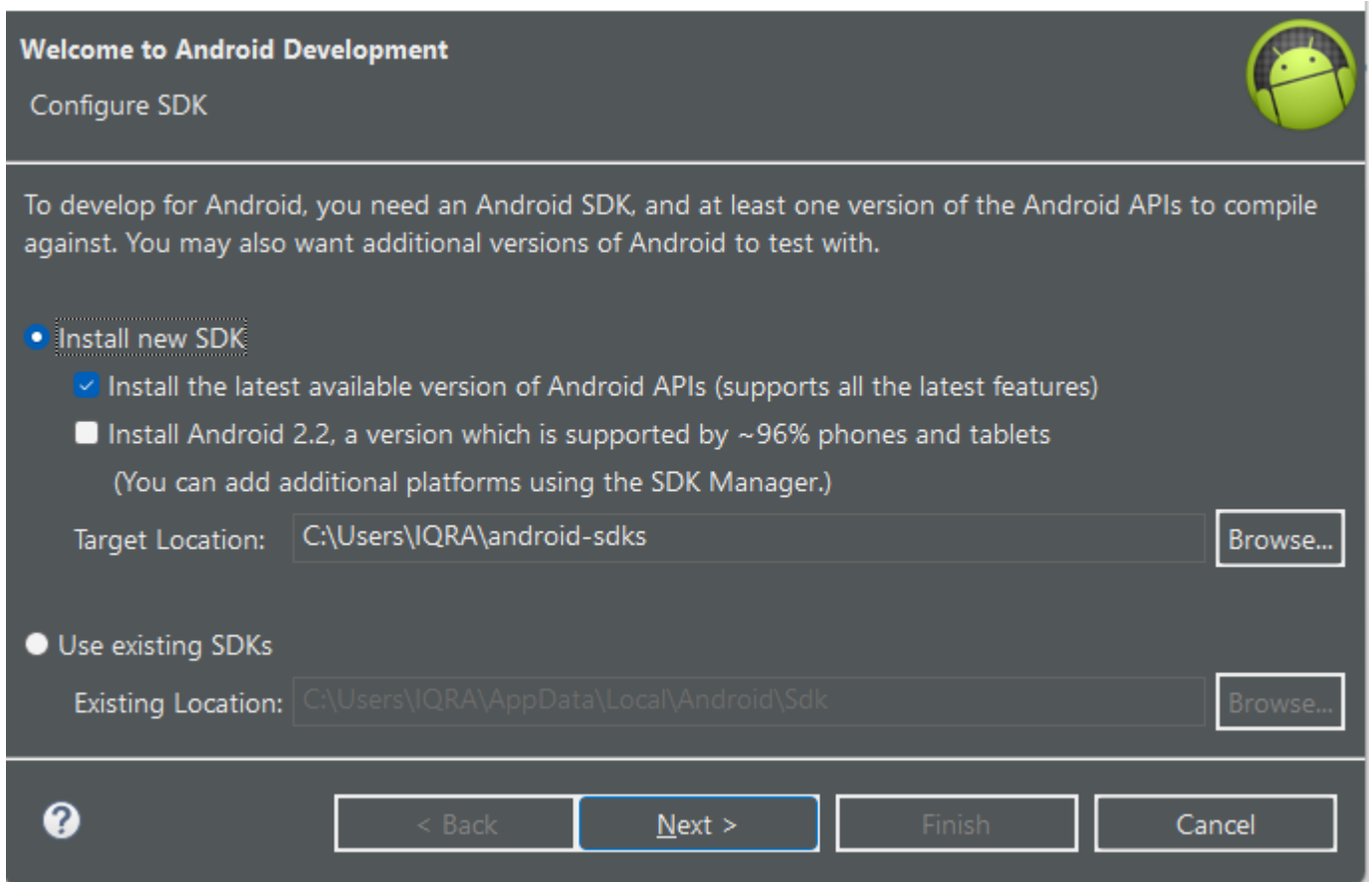


Step 6: Browse the SDK Location of Android (C:\Program Files\Android\android-sdk) and Click Apply.

Note 1: SDK Path is also present in Android Studio -> Tools -> SDK Manager -> Copy the Android SDK Location path and paste it here.

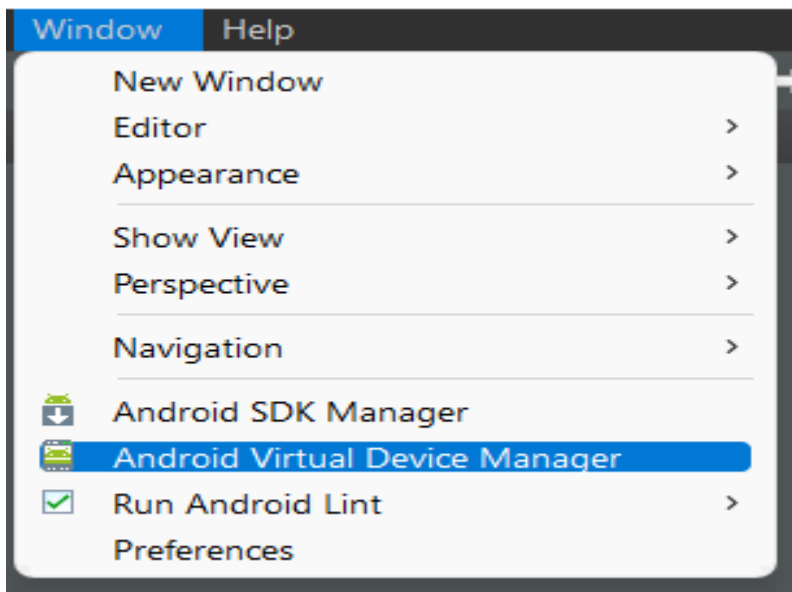


Step 7: Click on Install new SDK then Next. Another dialog box will appear, Accept all three packages and Click on Install.

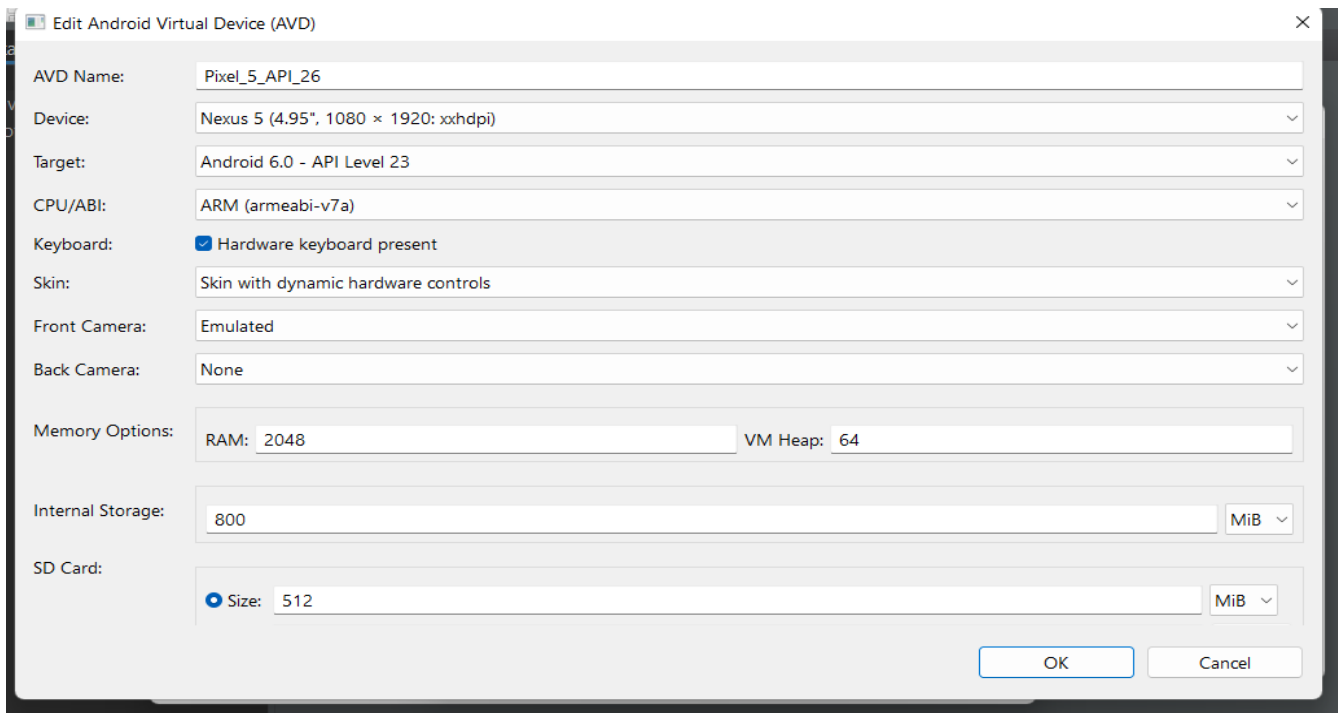


After installation of packages, SDK Manager will appear for API Level Build Tools and System Images installation. Click on Install.

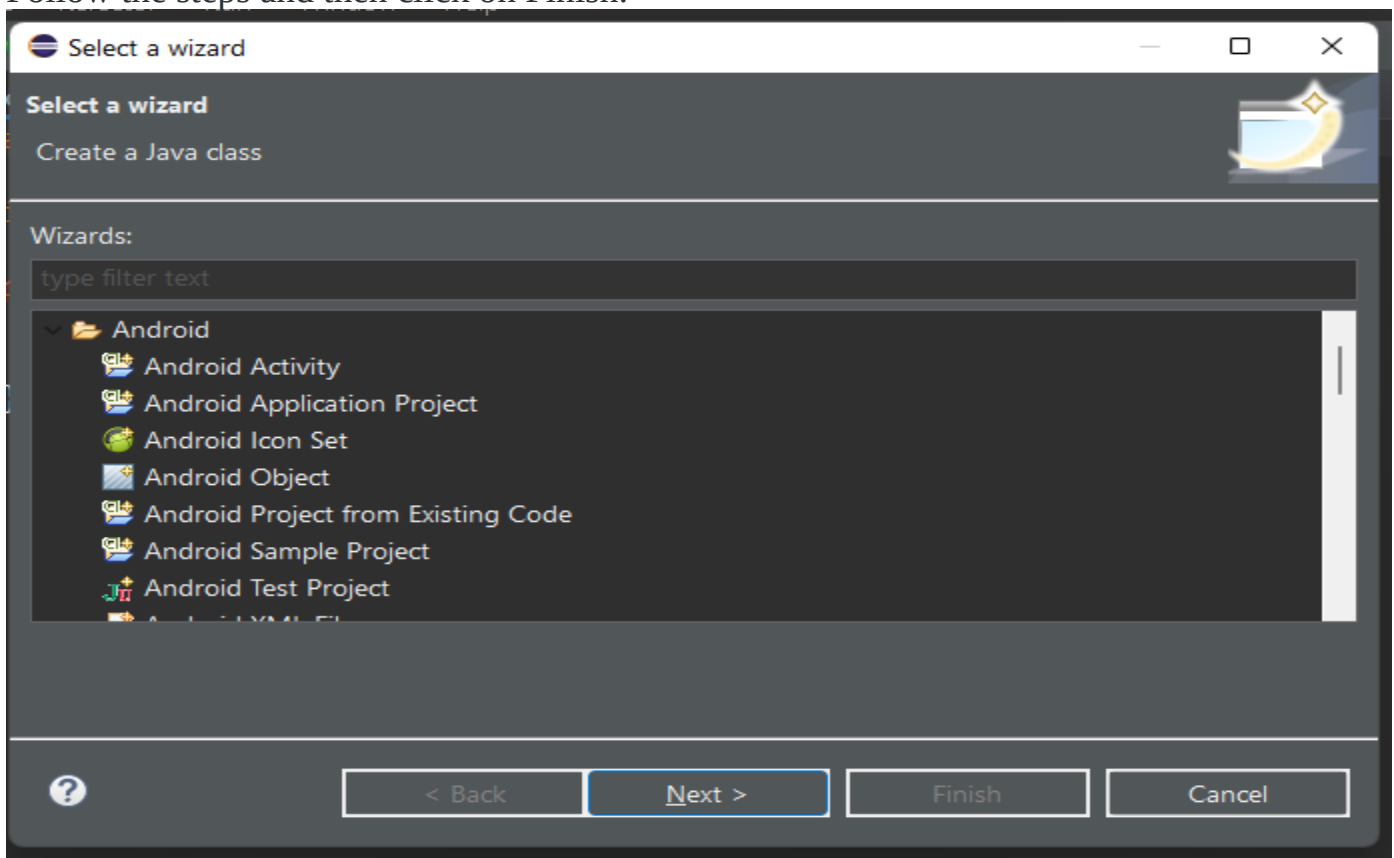
Step 8: After installation is completed, Go to Eclipse then Select Window then Click on Android Virtual Device Manager. A dialog box will appear, Select existing AVD and Click on Edit.



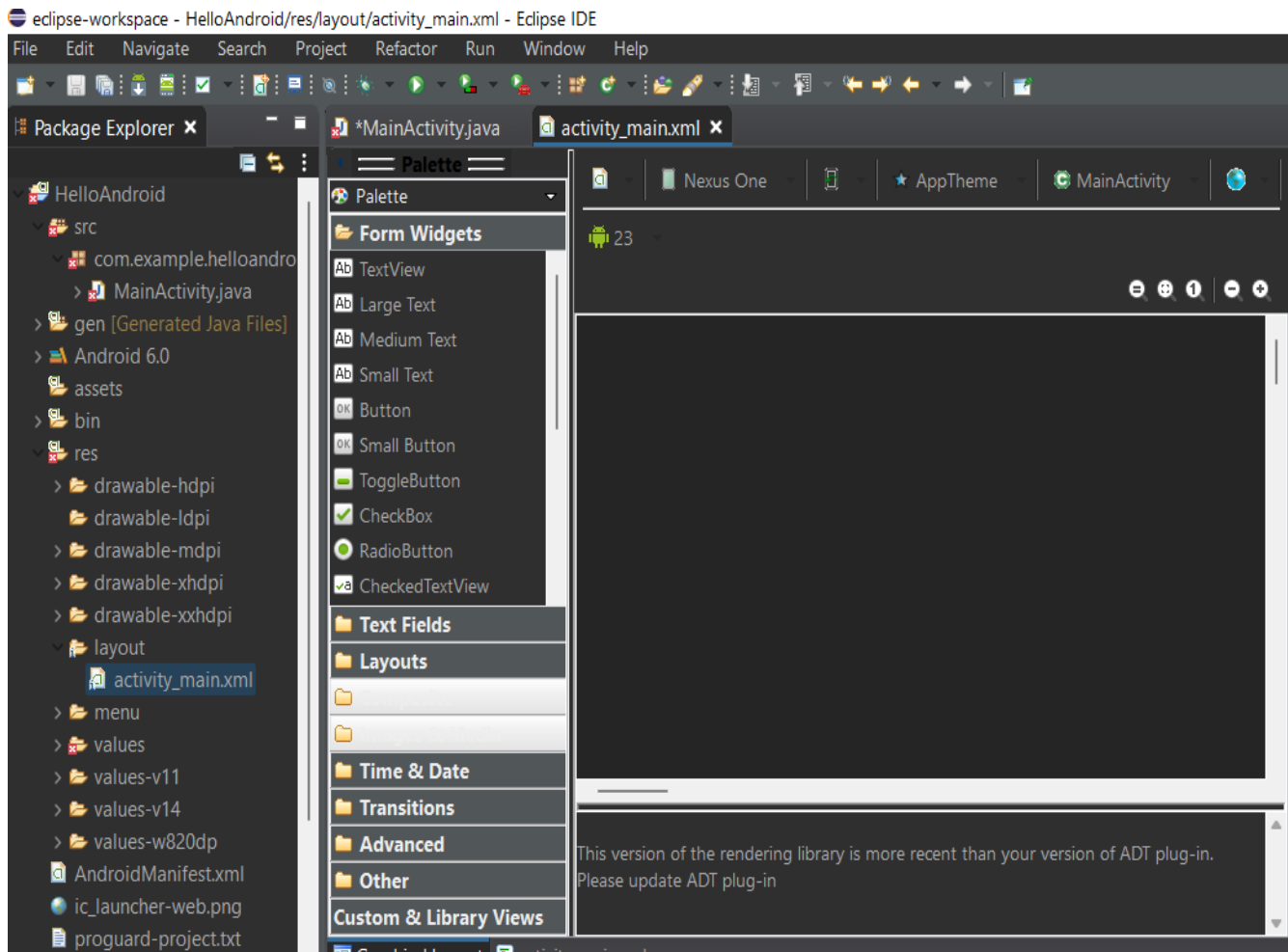
Fill in all the details as per the below image. Click OK.



Step 9: To create an android application, Select File -> New -> Other, and then the below dialog box will appear Select Android -> Android Application Project then Click on Next. Follow the steps and then click on Finish.



Step 10: Eclipse IDE setup is completed for Android Application Development.

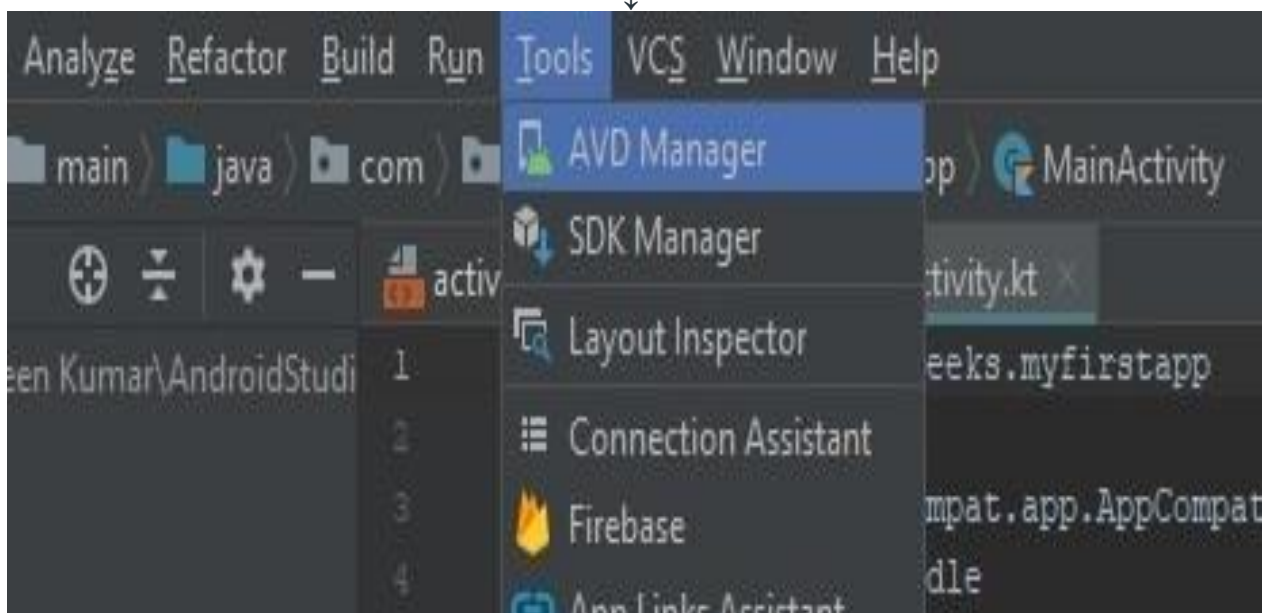
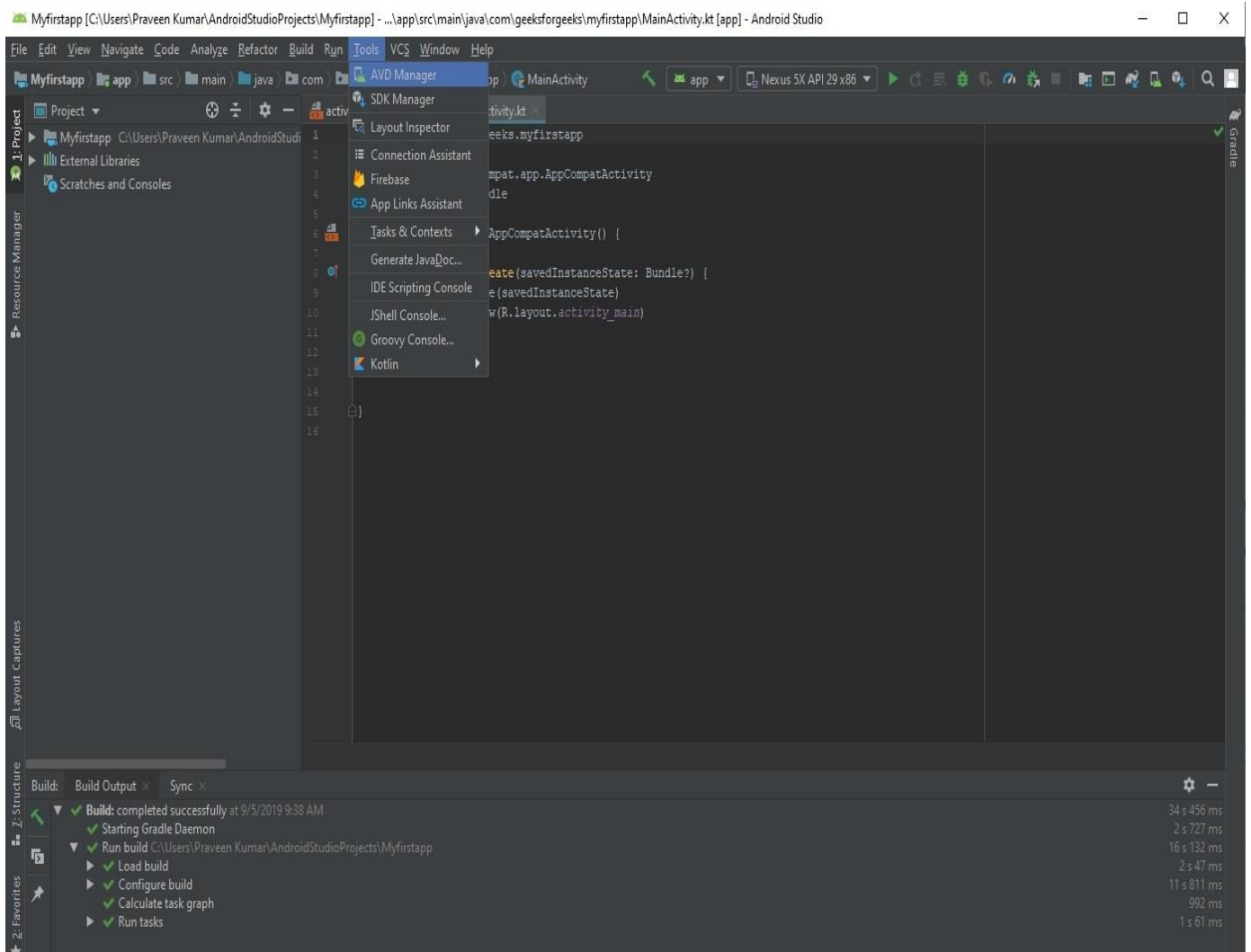


4. Create Android Virtual Device(AVD)

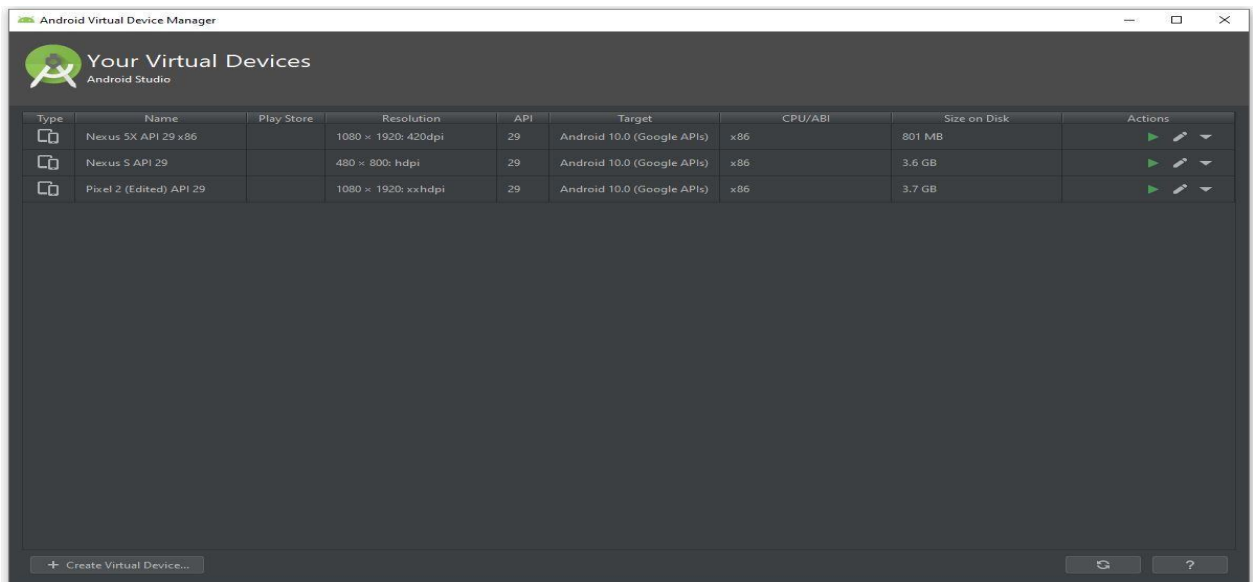
Follow the below steps to install **Android Virtual Device**.

Step 1: Go to Tools > AVD

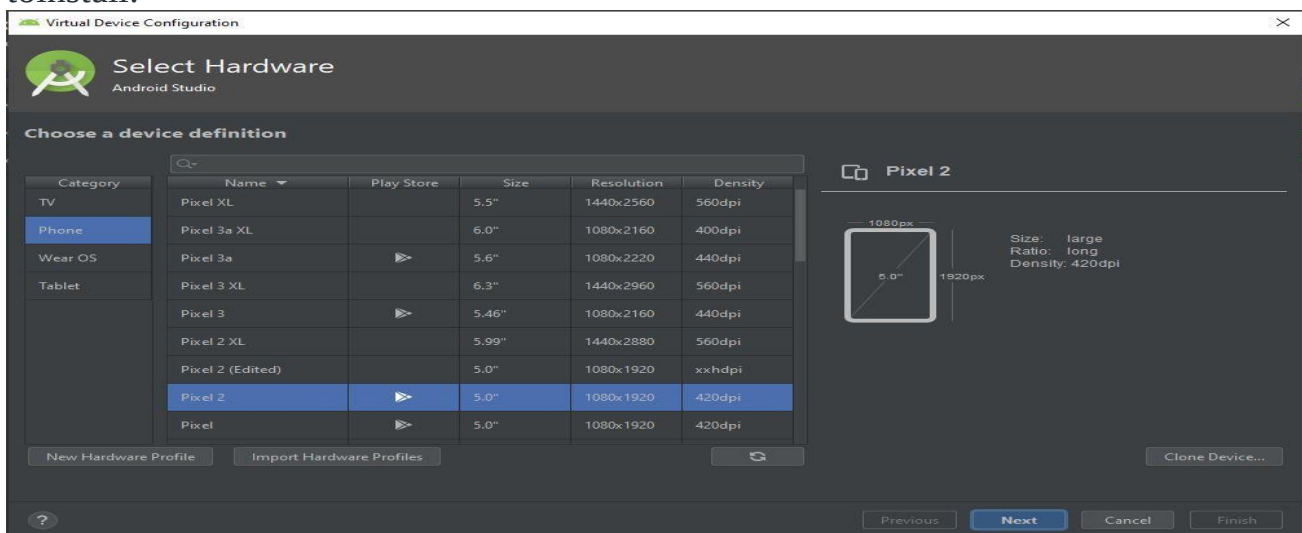
Manager.



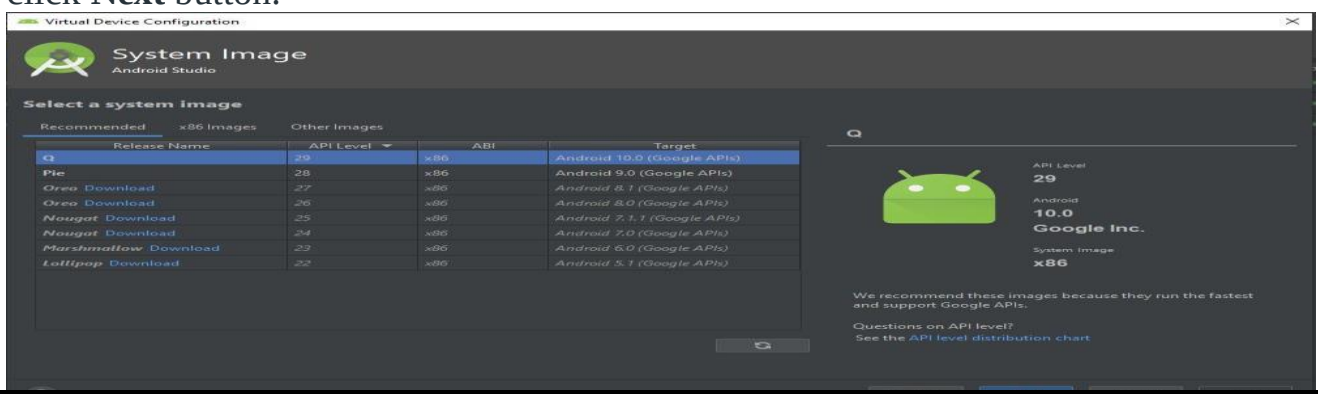
Step 2: Now click on Create Virtual Device.



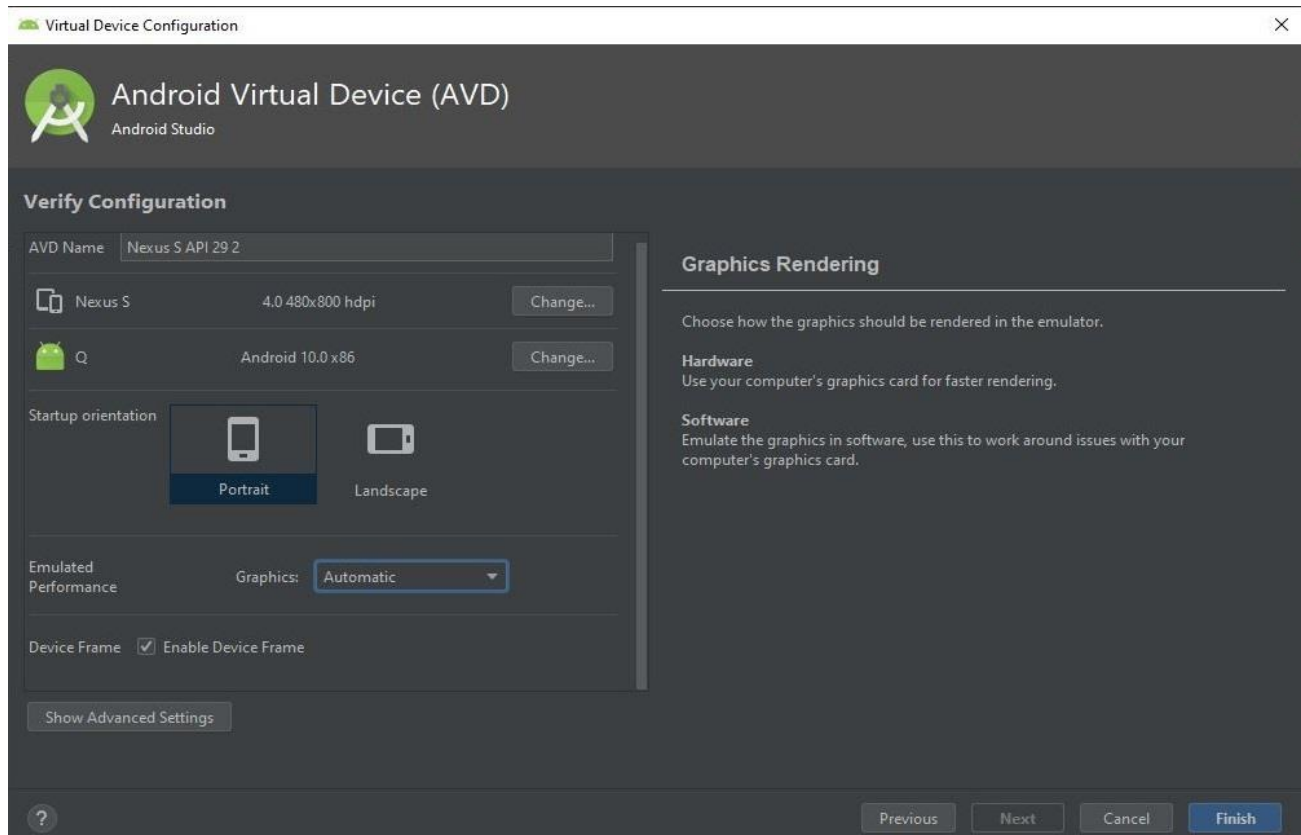
Step 3: A pop-up window will be there and here we select the category Phone because we are creating android app for mobile and select the model of mobile phone we want to install.



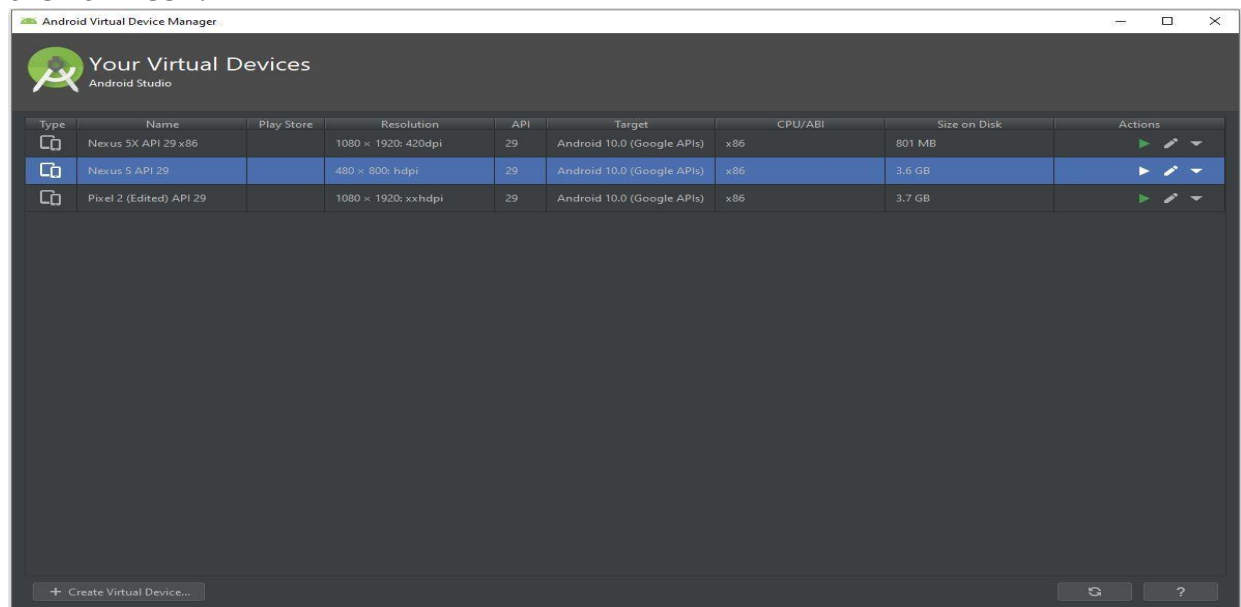
Step 4: Here we select the android version to download like Q, Pie, Oreo etc and click Next button.



Step 5: Click the finish button to complete the installation.



Step 6: Now we can select the virtual device we want to run as emulator can click on the **run** icon.



Step 7: Finally our virtual device is ready to run our android



Conclusion:

The Android Development Tools (ADT) plugin has been successfully installed and configured in Eclipse. With ADT, Eclipse is now fully equipped to support Android application development, including project creation, code editing, debugging, and testing. The integration of ADT with the Android SDK allows developers to efficiently build, run, and test Android applications using emulators or real devices. This setup provides a complete development environment for designing, developing, and deploying Android applications.

Submitted By:

Checked By : Ms.Swati.S.Badhe

Sign :

Name :

Roll No :

Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)

Practical: 02

DOP:

DOC:

Title: Create “Hello World” application. That will display “Hello World” in the middle of the screen using TextView Widget in the red color.

Objective : To understand the basic structure of an Android application and to learn how to use a TextView widget to display text on the screen with specific properties such as text content, color, and alignment by creating a simple “Hello World” application.

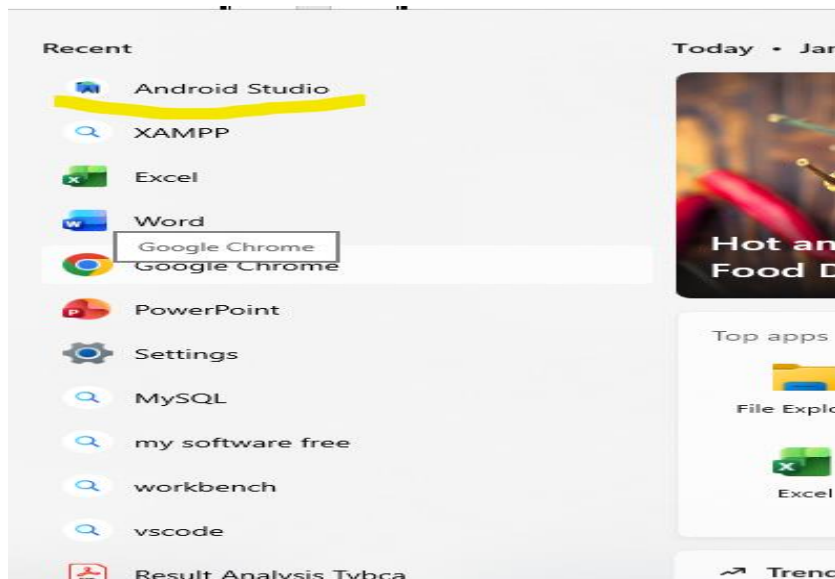
Requirements:

1. Computer or Laptop
2. Android Studio IDE
3. Java or Kotlin Programming Language
4. Android SDK
5. Android Emulator or Android Mobile Device
6. Basic knowledge of Android application development

Step-by-Step Instructions

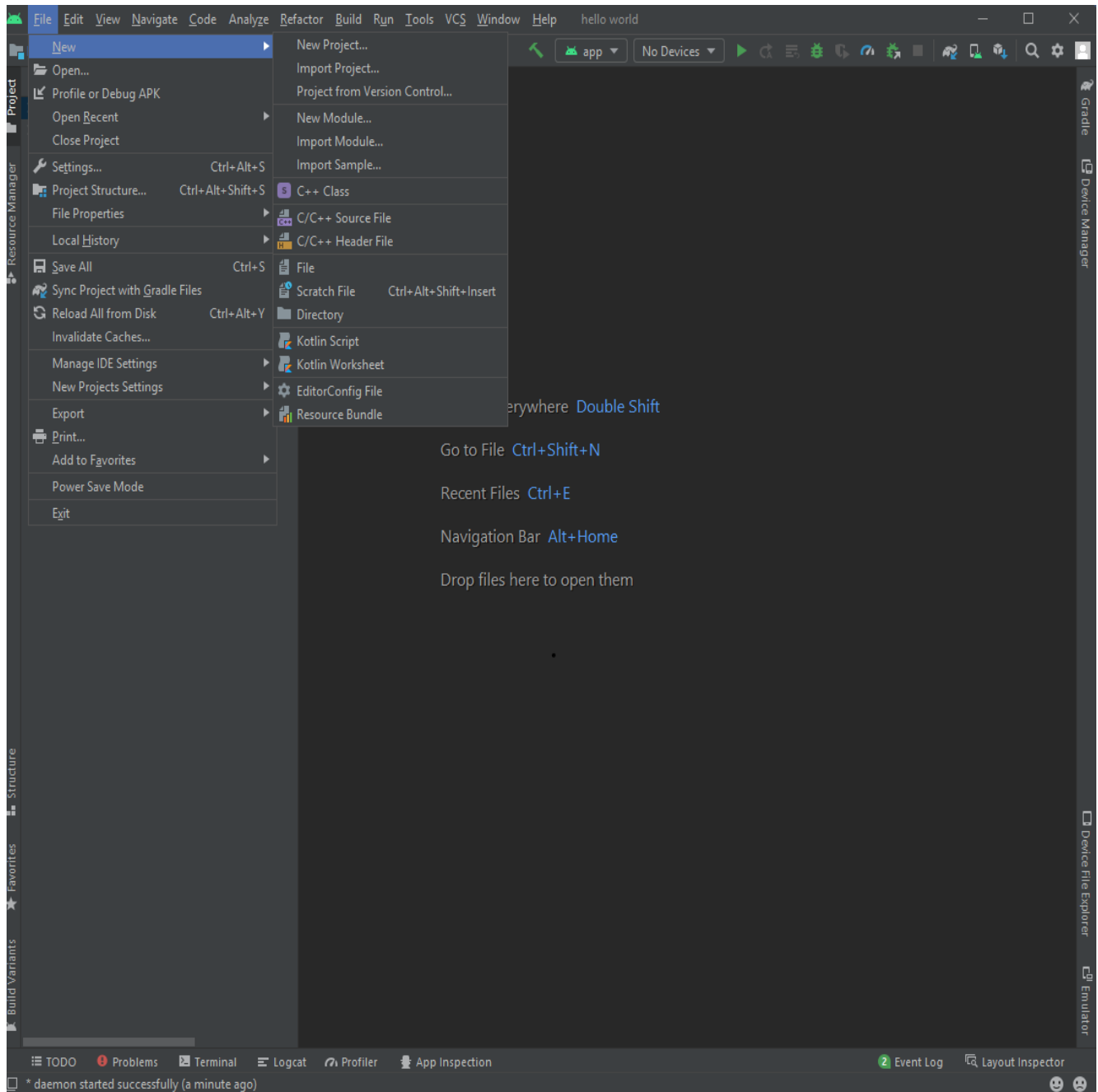
Step 1: Open Android Studio

1. Click on the Start Menu (Windows).
2. Search for Android Studio.
3. Click on Android Studio to open it.
4. Wait until the Welcome to Android Studio screen appears.



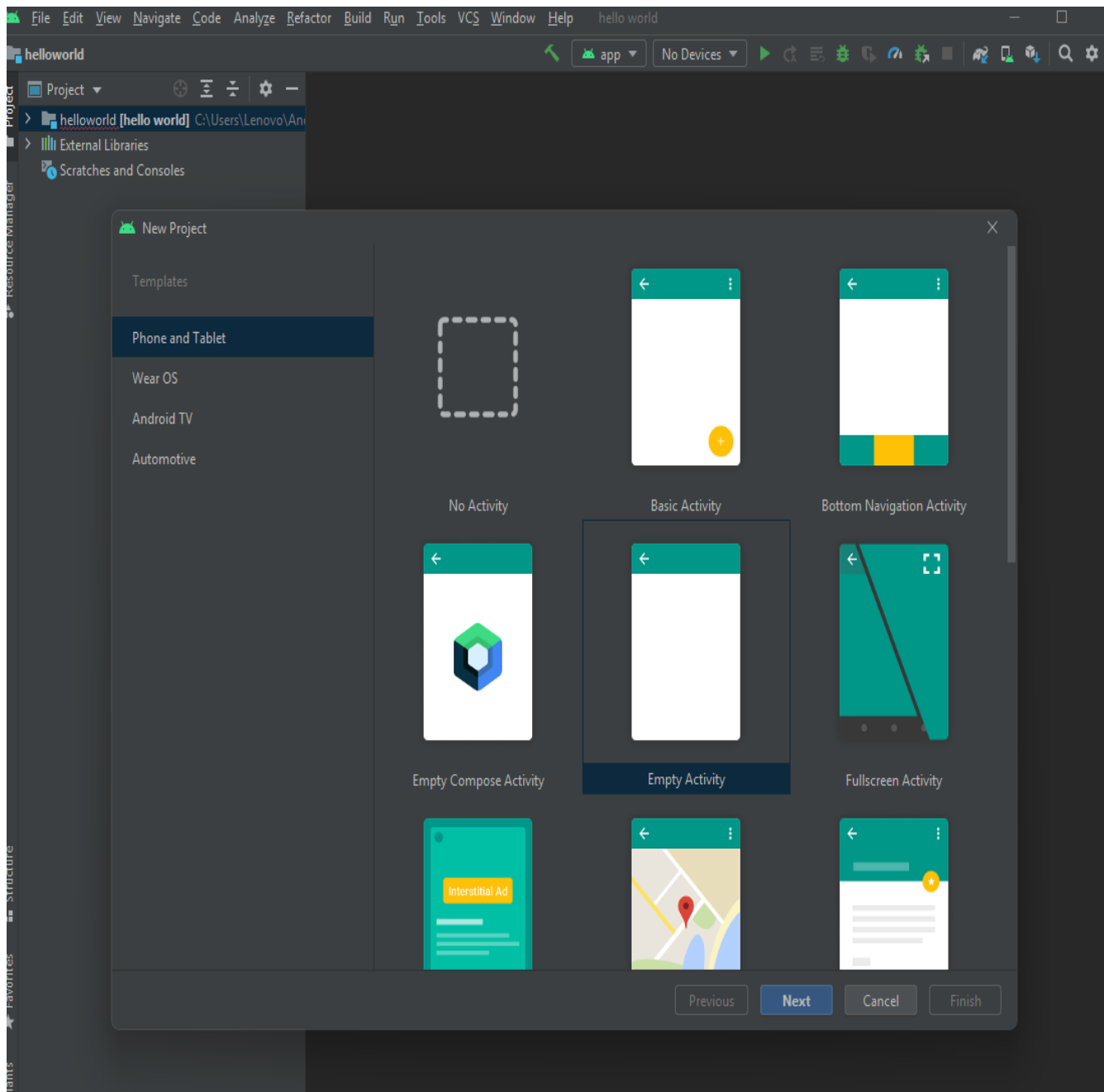
Step 2: Create a New Project

1. On the Welcome screen, click on **New Project**.
2. A list of project templates will be displayed.



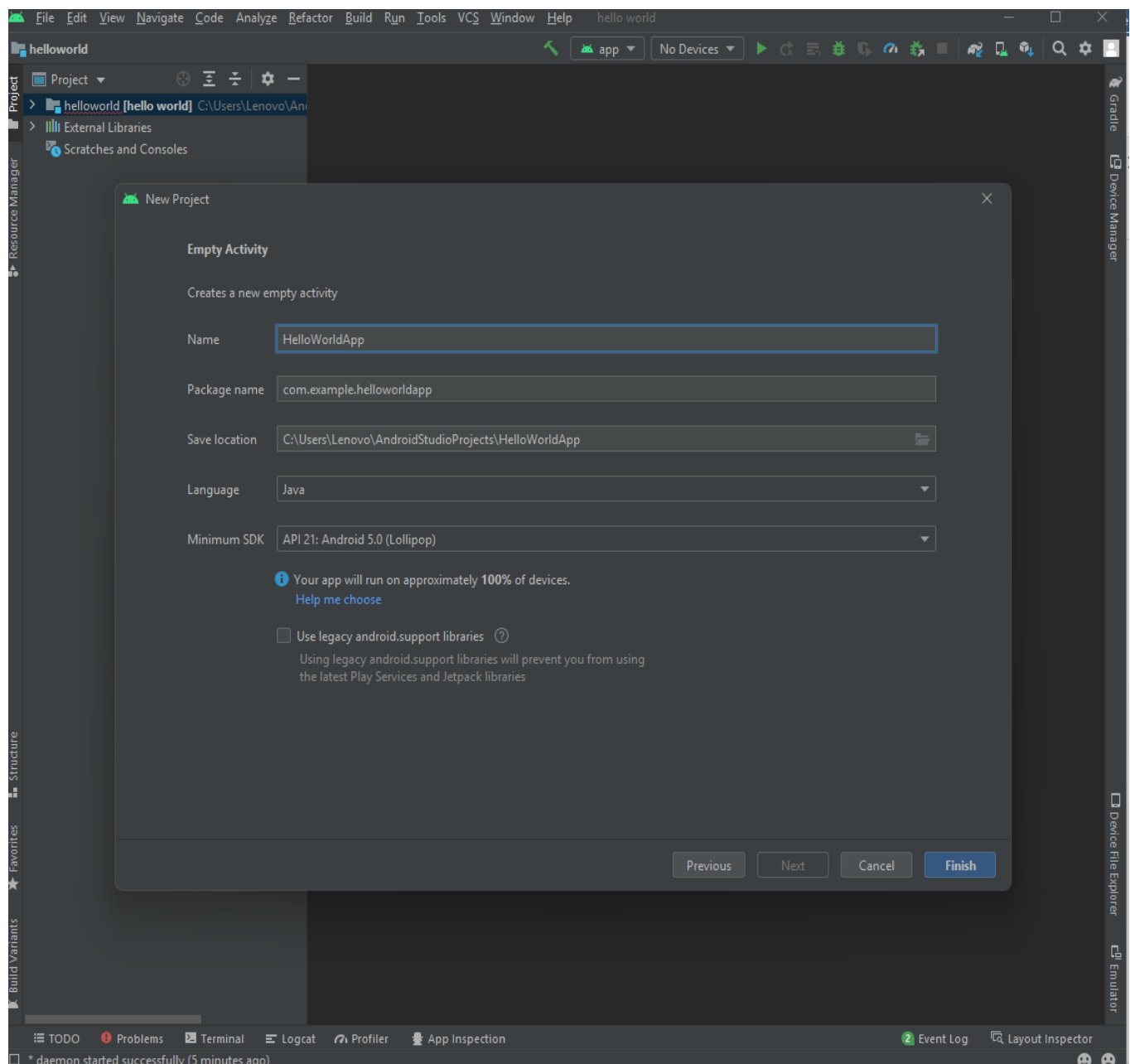
Step 3: Select Project Template

1. Select Empty Activity.
2. Click on the Next button.



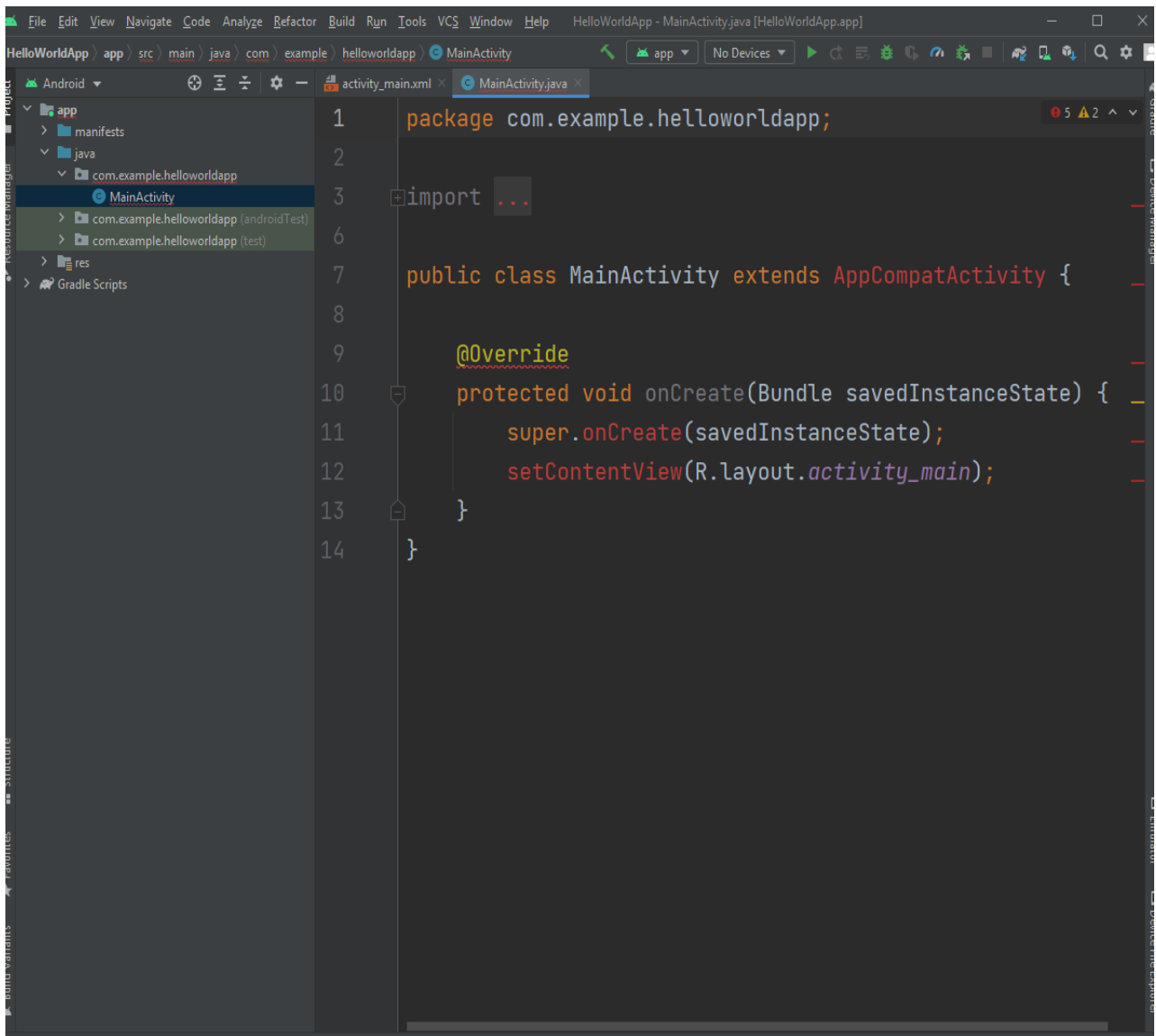
Step 4: Configure Your Project

1. Name: Enter the project name
Example: HelloWorldApp
2. Package Name: Auto-generated (leave as it is).
3. Save Location: Default location (leave as it is).
4. Language: Select Java.
5. Minimum SDK: Select API 21: Android 5.0 (Lollipop) or as required.
6. Click on the Finish button.



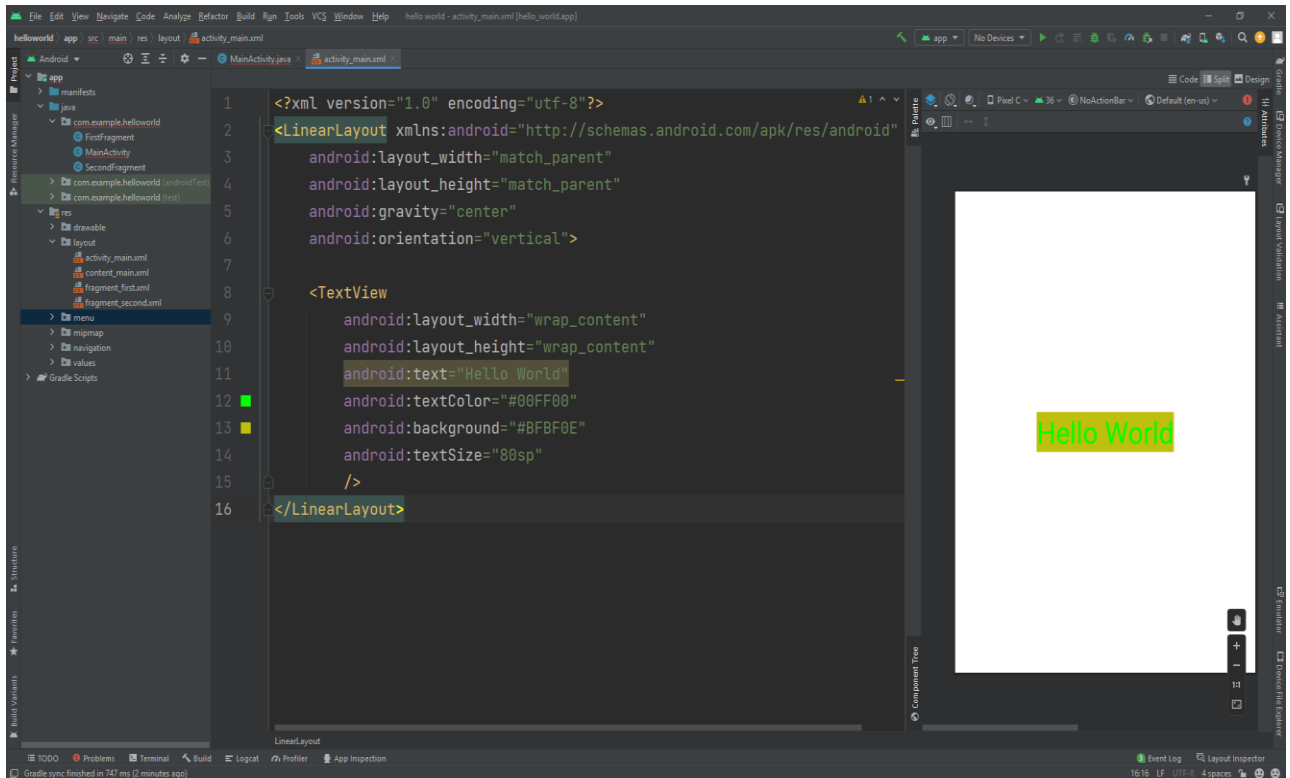
Step 5: Project Creation

1. Android Studio will start building the project.
2. Wait until **Gradle Build Finished** message appears at the bottom.
3. Your project is now successfully created.



Step 6: Open Layout File

1. Go to **app** → **res** → **layout**.
2. Open **activity_main.xml**.
3. Add or modify the **TextView** code as required.



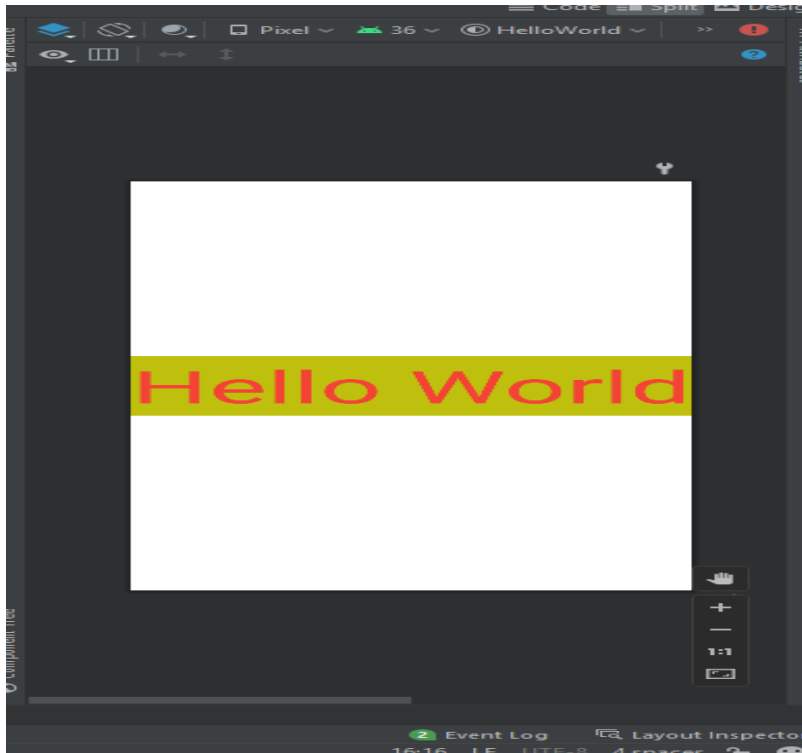
Android Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World"
        android:textColor="#00FF00"
        android:background="#BFBF0E"
        android:textSize="80sp"
    />

</LinearLayout>
```

Output:



Conclusion:

In this practical, a simple **“Hello World” Android application** was successfully created using **Android Studio**. The application demonstrates the basic structure of an Android app and the use of the **TextView widget** to display text. The text **“Hello World”** was displayed **at the center of the screen** with **red color**, **font size 60**, and a **background color**. This practical helped in understanding project creation steps, layout design using XML, and basic UI properties in Android application development.

Submitted By:

Checked By : Ms.Swati.S.Badhe

Sign :

Name:

Roll No :

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 03

DOP:

DOC:

Title: Create Registration page to demonstration of Basic widgets available in android.

Objective:

- To understand the use of **basic Android UI widgets**.
- To design a **user-friendly registration form** using XML.
- To learn how to use **EditText, Button, RadioButton, and CheckBox**.
- To understand **event handling** using button click listeners.
- To display messages using **Toast**.
- To gain hands-on experience in **Android layout design**.
- To understand the separation of **UI (XML)** and **logic (Kotlin/Java)**.

Requirements:

1. Android Studio (latest or any stable version)
2. Android SDK
3. JDK (Java Development Kit) / Kotlin support
4. Android Emulator or Physical Android Device
5. Operating System: Windows / Linux / macOS

Step-by-Step Instructions:

Step 1: Open Android Studio

- Click on the Start Menu (Windows).
- Search for Android Studio.
- Click on Android Studio to open it.
- Wait until the Welcome to Android Studio screen appears.

Step 2: Create a New Project

- On the Welcome screen, click on **New Project**.
- A list of project templates will be displayed.

Step 3: Select Project Template

- Select Empty Activity.
- Click on the Next button.

Step 4: Configure Your Project

- Name: Enter the project name
Example: HelloWorldApp
- Package Name: Auto-generated (leave as it is).
- Save Location: Default location (leave as it is).
- Language: Select Java.
- Minimum SDK: Select API 21: Android 5.0 (Lollipop) or as required.
- Click on the Finish button.

Step 5: Project Creation

- Android Studio will start building the project.
- Wait until **Gradle Build Finished** message appears at the bottom.
- Your project is now successfully created.

Step 6: Open Layout File

- Go to **app** → **res** → **layout**.
- Open **activity_main.xml**.
- Add or modify the **TextView code** as required.

Android Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
    <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <!-- Title -->
        <TextView
            android:id="@+id/tvTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Registration Form"
            android:textSize="24sp"
            android:textStyle="bold"
            android:layout_centerHorizontal="true"/>
```

<!-- Name Label -->

```
<TextView
    android:id="@+id/tvName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Name"
    android:textSize="20sp"
    android:textStyle="bold"
    android:layout_below="@id/tvTitle"
    android:layout_marginTop="15dp"/>
```

<!-- Name Input -->

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvName"/>
```

<!-- Address Label -->

```
<TextView
    android:id="@+id/tvAddress"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Address"
    android:textSize="19sp"
    android:textStyle="bold"
    android:layout_below="@id/etName"
    android:layout_marginTop="10dp"/>
```

<!-- Address Input -->

```
<EditText
    android:id="@+id/etAddress"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvAddress" />
```

<!-- City Label -->

```
<TextView
    android:id="@+id/tvCity"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="City"
    android:textSize="18sp"
```

```

        android:textStyle="bold"
        android:layout_below="@id/etAddress"
        android:layout_marginTop="10dp"/>
<!-- City Input -->
<EditText
    android:id="@+id/etCity"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvCity"/>
<!-- Submit Button -->
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:textSize="19sp"
    android:textStyle="bold"
    android:layout_below="@id/etCity"
    android:layout_marginTop="20dp"/>
</RelativeLayout>

```

Output:

Conclusion:

In this project, a simple Registration Form was successfully designed using RelativeLayout in Android. The form includes basic input fields such as Name, Address, and City, along with a Submit button. Large and bold text was applied to improve readability and user interaction. This project demonstrates the use of fundamental Android UI components like TextView, EditText, and Button, and helps in understanding layout management using RelativeLayout. The application provides a clear foundation for building more advanced Android forms in the future.

Submitted By:**Checked By: Ms.Swati.S.Badhe****Sign :****Name :****Roll No :**

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 04

DOP:

DOC:

**Title: Create sample application with login module.(Check username and password)
On successful login, Change TextView “Login Successful”. And on failing login, alert user using Toast “Login fail”.**

Objective :

The objective of this practical is to design and develop a sample Android application with a login module that validates the username and password entered by the user. On successful login, the application displays a TextView message “Login Successful”, and on failed login, it notifies the user with a Toast message “Login fail”. This practical helps in understanding user input handling, conditional checking, event handling, and basic UI interaction in Android applications.

Requirements:

1. Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM (recommended)
- Internet connection (for Android Studio setup)

2. Software Requirements

- Android Studio (latest version)
- Android SDK
- Java or Kotlin programming language
- Android Emulator or Android Mobile Device

3. Functional Requirements

- TextView for displaying messages
- EditText for entering username and password
- Button for Login
- Toast message for login failure
- Logic to validate username and password

4. Knowledge Requirements

- Basic understanding of Android UI components
- Knowledge of event handling
- Basic concepts of conditional statements

Android Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

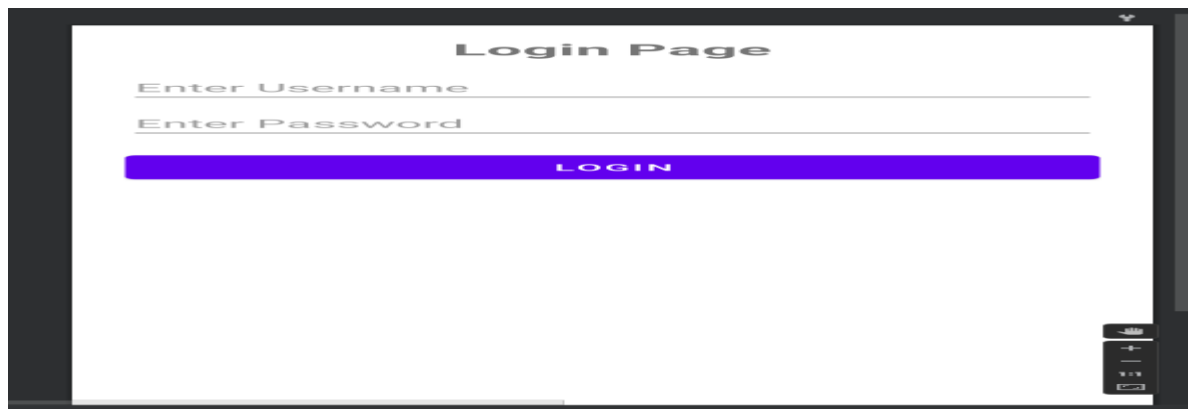
    <!-- Title -->
    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login Page"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_gravity="center"
        android:paddingBottom="20dp"/>

    <!-- Username -->
    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Username"/>

    <!-- Password -->
    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Password"
        android:inputType="textPassword"
        android:layout_marginTop="10dp"/>
```

```
<!-- Login Button -->
<Button
    android:id="@+id/btnLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"
    android:layout_marginTop="20dp"/>
</LinearLayout>
```

Output:



Conclusion:

This application is a simple Android login module using EditText, Button, TextView, and Toast. It checks the username and password entered by the user. If the login is correct, the TextView shows “Login Successful”; if it is wrong, a Toast message shows “Login fail”. This basic logic can later be extended to use databases, APIs, or shared preferences for more advanced login features.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign :

Name :

Roll No :

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 05

DOP:

DOC:

Title: Create an application for demonstration of Scroll view in android.

Objective : The objective of this practical is to demonstrate the use of ScrollView in Android, which allows content that is larger than the screen size to be scrollable. It helps in creating layouts where the user can scroll vertically to view all content.

Requirements:

1. Software Requirements

- Android Requirements
- Minimum API Level 21 (Lollipop)
- Android Emulator or a physical Android device for testing

2. Project Requirements

- New Android Project with Empty Activity
- Layout: Use ScrollView with LinearLayout inside
- Widgets: TextView, ImageView, Button (optional for demo)

Step-by-Step Instructions:

Android Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<ScrollView
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent">
```

```
    <LinearLayout
```

```
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
```

```
        android:orientation="vertical"
```

```
        android:padding="16dp">
```



```
<!-- Multiple TextViews to demonstrate scrolling -->
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 1"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 2"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 3"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<!-- Add more TextViews to exceed screen height -->
```

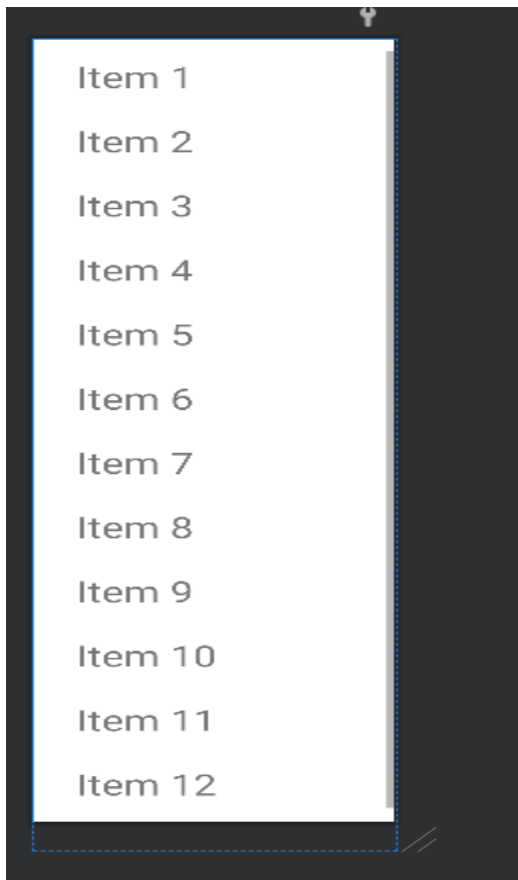
```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 4"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 5"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Item 6"  
    android:textSize="24sp"  
    android:padding="10dp"/>
```

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 7"
    android:textSize="24sp"
    android:padding="10dp"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 8"
    android:textSize="24sp"
    android:padding="10dp"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 9"
    android:textSize="24sp"
    android:padding="10dp"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 10"
    android:textSize="24sp"
    android:padding="10dp"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 11"
    android:textSize="24sp"
    android:padding="10dp"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Item 12"
    android:textSize="24sp"
    android:padding="10dp"/>
</LinearLayout>
</ScrollView>
```

Output:



Conclusion:

The practical demonstrates how to use **ScrollView** in Android to handle layouts larger than the screen. It ensures smooth vertical scrolling, allowing users to access all content in the app.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign :

Name :

Roll No :

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 06

DOP:

DOC:

Title: Create login application where you will have to validate username and passwords till the username and password is not validated, login button should remain disabled.

Objective :

The objective of creating this login application is to ensure that users enter valid username and password details before accessing the system. The login button remains disabled until both fields meet the required validation criteria, such as correct format and minimum length. This improves user experience by providing instant feedback, prevents invalid submissions, and enhances security by ensuring only properly validated data is processed.

Requirements:

- The application must have a username input field.
- The application must have a password input field.
- Both fields should not be empty.
- The username must follow the required format (e.g., email or minimum character length).
- The password must meet defined criteria (e.g., minimum length, special characters, etc.).
- The login button must be disabled by default.
- The login button should be enabled only when both username and password are valid.
- The system must provide real-time validation feedback.
- Appropriate error messages should be displayed for invalid inputs.
- The application must securely handle user credentials with proper validation.

Step-by-Step Instructions:

Android Layout File (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username"
        android:inputType="text" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"
        android:layout_marginTop="16dp" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login"
        android:enabled="false"
        android:layout_marginTop="24dp"/>
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.loginapp;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.widget.Button;
import android.widget.EditText;
```

```

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etUsername, etPassword;
    Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etUsername = findViewById(R.id.etUsername);
        etPassword = findViewById(R.id.etPassword);
        btnLogin = findViewById(R.id.btnLogin);

        TextWatcher textWatcher = new TextWatcher() {
            @Override
            public void beforeTextChanged(CharSequence s, int start, int count, int after) { }

            @Override
            public void onTextChanged(CharSequence s, int start, int before, int count) {
                validateInputs();
            }

            @Override
            public void afterTextChanged(Editable s) { }
        };

        etUsername.addTextChangedListener(textWatcher);
        etPassword.addTextChangedListener(textWatcher);
    }

    private void validateInputs() {
        String username = etUsername.getText().toString().trim();
        String password = etPassword.getText().toString().trim();

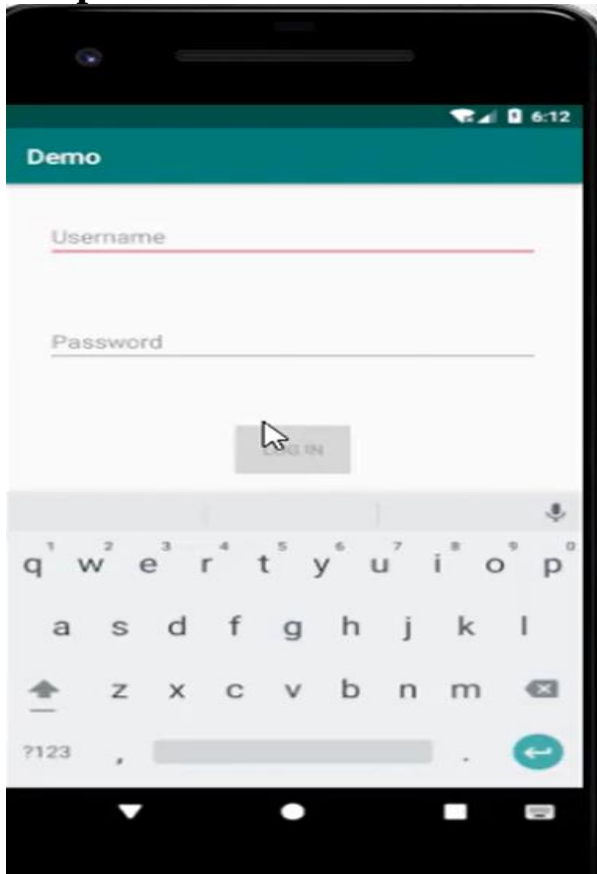
        boolean isValid = username.length() >= 4 && password.length() >= 6;

        btnLogin.setEnabled(isValid);
    }
}

```

}

Output:



Conclusion:

In this project, a Login Application was successfully developed with proper username and password validation functionality. The system ensures that the Login button remains disabled until valid credentials are entered, improving both security and user experience.

This implementation demonstrates the use of input validation, event handling, and dynamic UI control in application development. By enabling the login button only after successful validation, the application prevents incorrect or incomplete submissions and ensures controlled access.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign :

Name :

Roll No :

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 07

DOP:

DOC:

Title: Create an application for calculator

Objective : The objective of creating a calculator application is to provide a simple and efficient tool for performing basic arithmetic operations like addition, subtraction, multiplication, and division. The app should be user-friendly, accurate, and able to handle errors such as division by zero. It aims to deliver immediate results, offer a clear interface for easy input, and improve programming skills through the implementation of event handling, logical operations, and user interface design..

Requirements :

- Display area to show input and results.
- Buttons for digits (0–9) and basic operations (+, −, ×, ÷).
- Clear (C) button to reset input.
- Accurate calculation and immediate result display.
- Handle errors like division by zero.
- Simple and user-friendly interface.

Step-by-Step Instructions:

Activity_main.xml (res/layout/activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```



```
android:layout_height="match_parent"
```

```
android:orientation="vertical"
```

```
android:background="#F5F5F5"
```

```
android:padding="12dp">
```

```
<!-- Display -->
```

```
<TextView
```

```
    android:id="@+id/tvDisplay"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="80dp"
```

```
    android:background="#FFFFFF"
```

```
    android:textColor="#000000"
```

```
    android:textSize="28sp"
```

```
    android:gravity="center_vertical|end"
```

```
    android:padding="16dp"
```

```
    android:text="125+35 = 160" />
```

```
<GridLayout
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:columnCount="4"
```

```
    android:layout_marginTop="12dp">
```

<!-- Row 1 -->

<Button android:text="C" style="@style/BtnClearLight" android:onClick="clear"/>

<Button android:text="/" style="@style/BtnOpLight" android:onClick="input"/>

<Button android:text="×" style="@style/BtnOpLight" android:onClick="input"/>

<Button android:text="-" style="@style/BtnOpLight" android:onClick="input"/>

<!-- Row 2 -->

<Button android:text="7" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="8" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="9" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="+" style="@style/BtnOpLight" android:onClick="input"/>

<!-- Row 3 -->

<Button android:text="4" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="5" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="6" style="@style/BtnNumLight" android:onClick="input"/>

<!-- Row 4 -->

<Button android:text="1" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="2" style="@style/BtnNumLight" android:onClick="input"/>

<Button android:text="3" style="@style/BtnNumLight" android:onClick="input"/>

<!-- Row 5 -->

<Button android:text="0" style="@style/BtnNumLight" android:onClick="input"/>

```
<Button android:text="." style="@style/BtnNumLight" android:onClick="input"/>
```

```
<Button
```

```
    android:text="="
```

```
    android:layout_columnSpan="2"
```

```
    style="@style/BtnEqualLight"
```

```
    android:onClick="calculate"/>
```

```
</GridLayout>
```

```
</LinearLayout>
```

styles.xml (res/values/styles.xml)

```
<resources>
```

```
    <style name="BtnNumLight">
```

```
        <item name="android:layout_width">0dp</item>
```

```
        <item name="android:layout_height">70dp</item>
```

```
        <item name="android:layout_columnWeight">1</item>
```

```
        <item name="android:textSize">20sp</item>
```

```
        <item name="android:background">#FFFFFF</item>
```

```
        <item name="android:textColor">#000000</item>
```

```
        <item name="android:layout_margin">4dp</item>
```

```
        <item name="android:elevation">4dp</item>
```

```
    </style>
```

```
<style name="BtnOpLight" parent="BtnNumLight">
```

```
    <item name="android:textColor">#1976D2</item>
```

```
</style>
```

```
<style name="BtnClearLight" parent="BtnNumLight">
```

```
    <item name="android:background">#E53935</item>
```

```
    <item name="android:textColor">#FFFFFF</item>
```

```
</style>
```

```
<style name="BtnEqualLight" parent="BtnNumLight">
```

```
    <item name="android:background">#43A047</item>
```

```
    <item name="android:textColor">#FFFFFF</item>
```

```
</style>
```

```
</resources>
```

MainActivity.java(java/com/example/calculator/MainActivity.java)

```
package com.example.calculator;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.view.View;
```

```
import android.widget.Button;
```

```
import android.widget.TextView;
```

```
public class MainActivity extends AppCompatActivity {
```

```

TextView tvDisplay;

String expression = "";

@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    tvDisplay = findViewById(R.id.tvDisplay);    }

public void input(View view) {

    Button b = (Button) view;

    expression += b.getText().toString();

    tvDisplay.setText(expression); }

public void clear(View view) {

    expression = "";

    tvDisplay.setText("");

}

public void calculate(View view) {

    try {

        expression = expression.replace("×", "*");

        double result = eval(expression);

        tvDisplay.setText(expression + " = " + result);

```

```

        expression = "";
    } catch (Exception e) {

        tvDisplay.setText("Error");

        expression = "";
    } }

// Simple evaluator

private double eval(String exp) {

    return new Object() {

        int pos = -1, ch;

        void nextChar() {

            ch = (++pos < exp.length()) ? exp.charAt(pos) : -1;
        }

        boolean eat(int charToEat) {

            while (ch == ' ') nextChar();

            if (ch == charToEat) {

                nextChar();

                return true; }

            return false;
        }

        double parse() {  nextChar();

```

```

double x = parseExpression();

return x;      }

double parseExpression() {

    double x = parseTerm();

    for (;;) {

        if (eat('+')) x += parseTerm();

        else if (eat('-')) x -= parseTerm();

        else return x;

    } }

double parseTerm() {

    double x = parseFactor();

    for (;;) {

        if (eat('*')) x *= parseFactor();

        else if (eat('/')) x /= parseFactor();

        else return x;

    }}

double parseFactor() {

    if (eat('+')) return parseFactor();

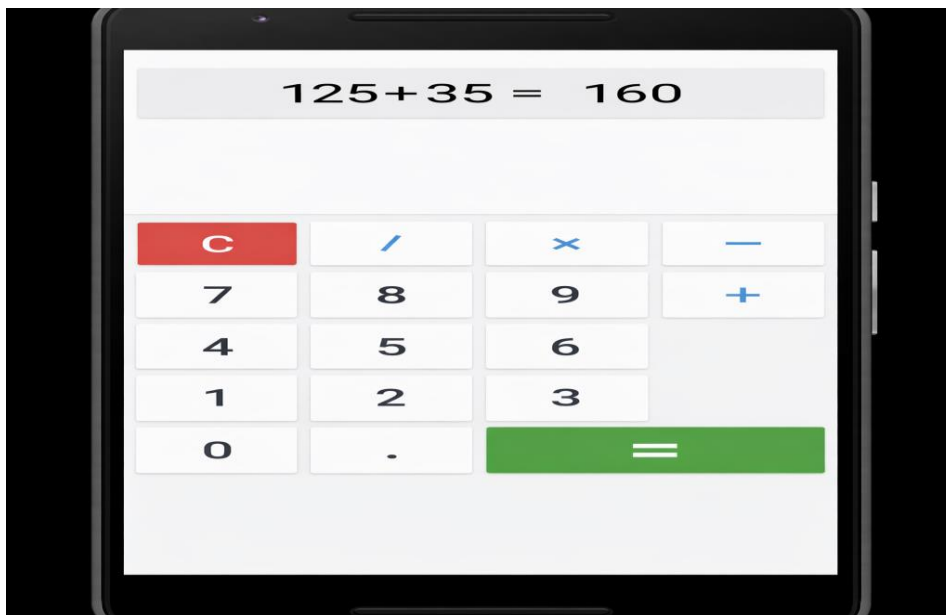
    if (eat('-')) return -parseFactor();

    double x;   int startPos = pos;

```

```
while ((ch >= '0' && ch <= '9') || ch == '.') nextChar();  
  
    x = Double.parseDouble(exp.substring(startPos, pos));  
  
    return x;  
  
} }.parse();  
  
} }
```

Output



Conclusion:

In this project, a simple Android Calculator application was successfully designed and implemented using Java and XML in Android Studio. The application performs basic arithmetic operations such as addition, subtraction, multiplication, and division with an easy-to-use and attractive light theme user interface.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign :

Name :

Roll No :

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 08

DOP:

DOC:

Title: Create an application for demonstration of Scroll view in android.

Objective: The objective of this lab manual is to guide you through the process of creating an Android application that demonstrates the usage of a Scroll View. Scroll View allows users to scroll through content that is larger than the physical screen size, making it useful for displaying lengthy content.

Theory:

1. Create a New Android Project:

- Open Android Studio.
- Click on "Start a new Android Studio project" or go to File > New > New Project.
- Enter the application name as "Scroll View Demo".
- Choose "Phone and Tablet" as the form factor and set the minimum SDK to API 16 or higher.
- Select "Empty Activity" as the template and click "Finish".

2. Design the Layout with Scroll View:

- Open the activity_main.xml file located in the "res/layout" directory.
- Replace the existing code with the following XML code:

3. Create String Resource for Lorem Ipsum Text:

- Open the strings.xml file located in the "res/values" directory.
- Add the following string resource:

4. Testing the Scroll View Application:

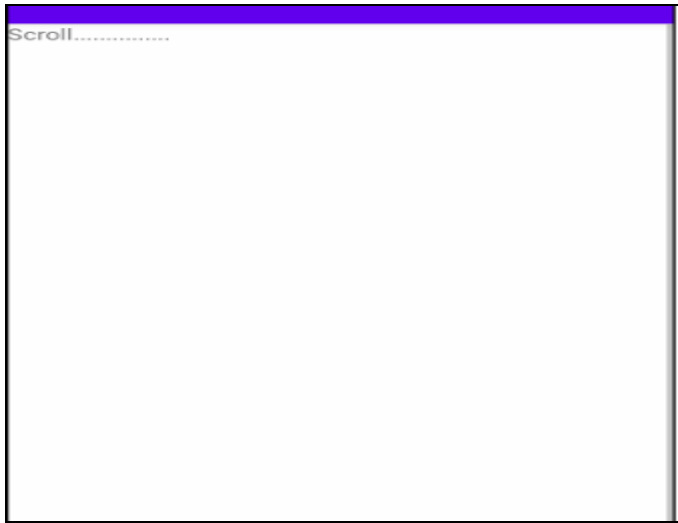
- Connect your Android device or launch the AVD.
- Build and run the application by clicking the green "play" button in Android Studio.
- Select your device or AVD from the deployment target list and click "OK".

- The application should launch on the device/emulator, displaying the Lorem Ipsum text within a Scroll View.
- Verify that you can scroll through the content vertically to view all the text.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    >
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Scroll....."/>
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="April Fool:)"
            android:layout_gravity="center_horizontal"
            android:layout_marginTop="500pt"
            />
    </LinearLayout>
</ScrollView>
```

Output:



Conclusion: This application demonstrates the use of Scroll View to manage content that exceeds the screen size. Scroll View is essential for creating scrollable layouts, especially for forms, articles, or large data displays

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign:

Name:

Roll No:

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 09

DOP:

DOC:

Title: Demonstrate use of intent in android

Objective: To understand and implement **Intent** in Android to move from one Activity to another and pass data between screens using **Android Studio**.

Requirements:

- Android Studio
- Java/Kotlin knowledge
- Android SDK
- Emulator or Android device

MainActivity.java

```
package com.example.intentdemo;
```

```
import android.content.Intent;  
import android.net.Uri;  
import android.os.Bundle;  
import android.view.View;  
import android.widget.Button;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
public class MainActivity extends AppCompatActivity {
```

```
    Button btnExplicit, btnImplicit;
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);
```

```
        btnExplicit = findViewById(R.id.btnExplicit);  
        btnImplicit = findViewById(R.id.btnImplicit);
```

```
        // Explicit Intent (Open Second Activity)
```

```

btnExplicit.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View v) {
        Intent intent = new Intent(MainActivity.this, SecondActivity.class);
        intent.putExtra("message", "Welcome to Second Activity");
        startActivity(intent);
    }
});

// Implicit Intent (Open Web Browser)
btnImplicit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse("https://www.google.com"));
        startActivity(intent);
    }
});

```

SecondActivity.java

```

package com.example.intentdemo;

import android.os.Bundle;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {

    TextView textView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        textView = findViewById(R.id.textView);

        String msg = getIntent().getStringExtra("message");
        textView.setText(msg);
    }
}

```

activity_main.xml

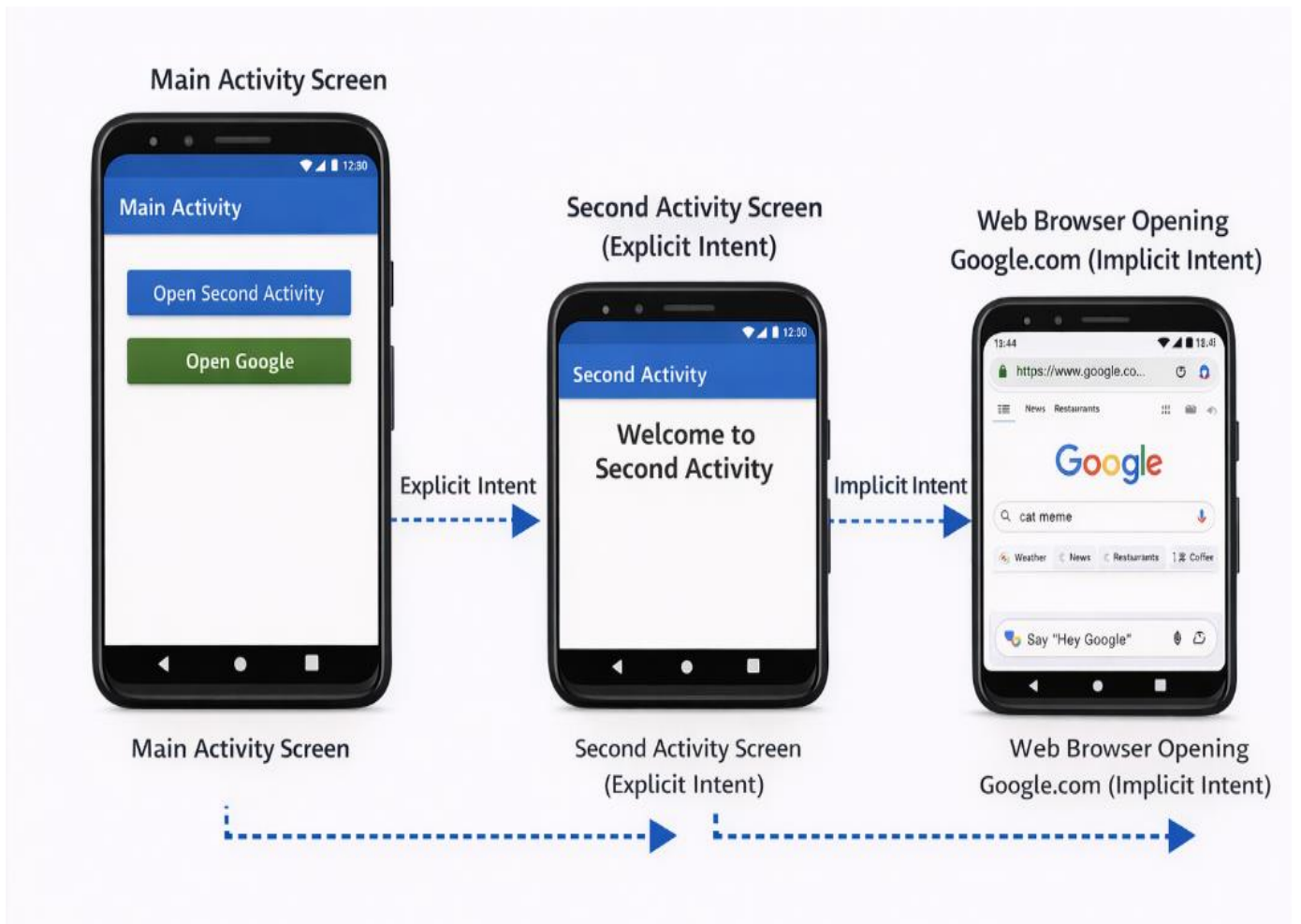
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android=http://schemas.android.com/apk/res/android
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <Button
        android:id="@+id/btnExplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Second Activity"/>
    <Button
        android:id="@+id/btnImplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Google"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp"/>
</LinearLayout>
```

Output:

When you click Open Second Activity → Second screen opens (Explicit Intent).
When you click Open Google → Web browser opens Google (Implicit Intent).



Conclusion: This practical demonstrates how to use **Intent** in Android to navigate between Activities using Explicit Intent and to perform actions like opening a web browser using Implicit Intent. It helps in understanding screen navigation and communication between components in Android applications.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign:

Name:

Roll No:

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 10

DOP:

DOC:

Title: Create application to demonstrate menu option.

Objective: To develop an Android application that demonstrates the use of Menu options (Options Menu) to perform different actions when menu items are selected using **Android Studio**.

Requirements:

- Android Studio
- Java/Kotlin knowledge
- Android SDK
- Emulator or Android device

MainActivity.java

```
package com.example.menudemo;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    // Create Menu
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    // Handle Menu Click
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
```



```

int id = item.getItemId();
    if (id == R.id.menu_settings) {
        Toast.makeText(this, "Settings Clicked", Toast.LENGTH_SHORT).show();
        return true;
    }
    if (id == R.id.menu_about) {
        Toast.makeText(this, "About Clicked", Toast.LENGTH_SHORT).show();
        return true;
    }
    if (id == R.id.menu_exit) {
        finish();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
}

```

activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Menu Demonstration App"
        android:textSize="20sp"/>
</LinearLayout>

```

Create Menu Folder

Right click → **res** → New → Android Resource Directory →

Select **Resource type = menu**

menu_main.xml (res/menu/menu_main.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/menu_settings"
        android:title="Settings"/>
    <item
        android:id="@+id/menu_about"
        android:title="About"/>

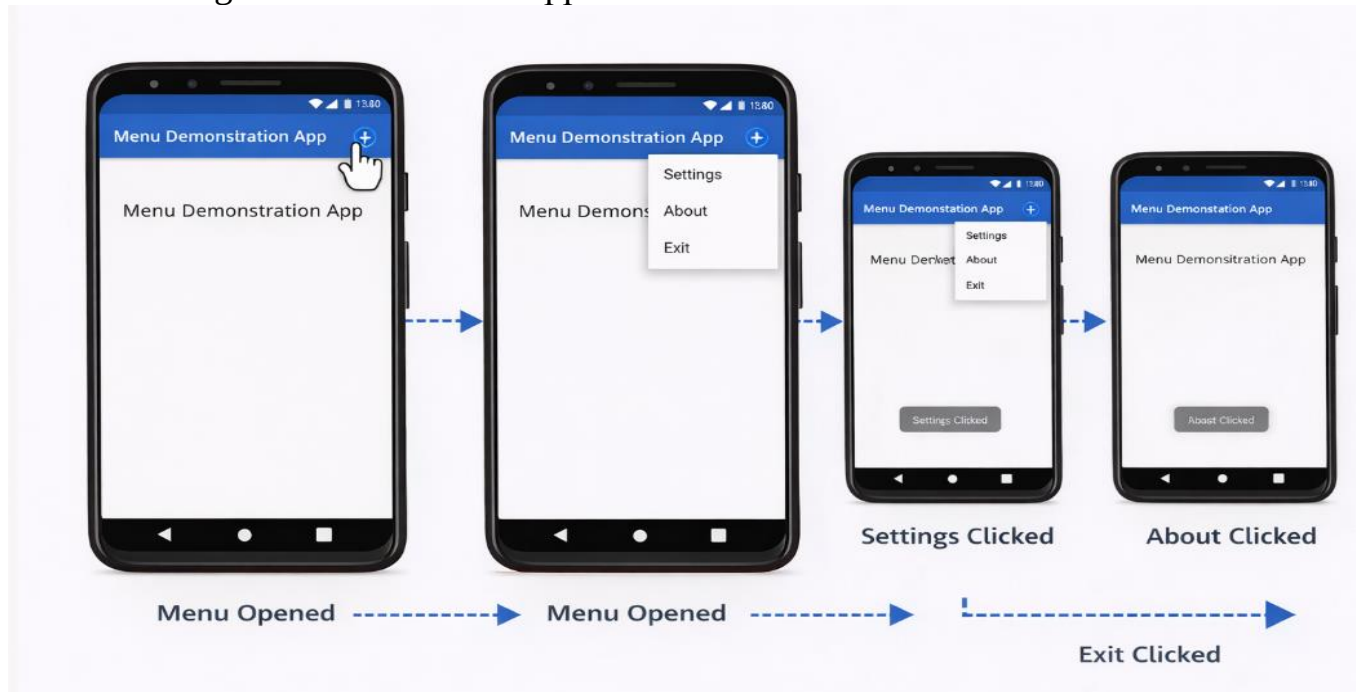
```

```
<item
    android:id="@+id/menu_exit"
    android:title="Exit"/>
</menu>
```

Output

When you click the **three dots (:)** menu:

- Clicking **Settings** → Shows Toast "Settings Clicked"
- Clicking **About** → Shows Toast "About Clicked"
- Clicking **Exit** → Closes the application



Conclusion: This practical demonstrates how to create and implement an Options Menu in an Android application. It helps in understanding how to add menu items and handle user selections using menu methods. Menu options improve user interaction and make the application more user-friendly and organized.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign:

Nam:

Roll No:

**Shram Sadhana Bombay Trust Sanchlit
Arts, Commerce & Science, Bambhori, college, Jalgaon
Bachelor Of Computer Application (B.C.A)**

Practical: 11

DOP:

DOC:

Title: .Create application to demonstrate progress bar.

Objective: To develop an Android application that demonstrates the use of a Progress Bar to show task progress or loading status using Android Studio.

Requirements:

- Android Studio
- Java/Kotlin knowledge
- Android SDK
- Emulator or Android device

MainActivity.java

```
package com.example.progressdemo;

import android.os.Bundle;
import android.os.Handler;
import android.widget.Button;
import android.widget.ProgressBar;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    ProgressBar progressBar;

    Button btnStart;

    int progressStatus = 0;

    Handler handler = new Handler();

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);
```

```

setContentView(R.layout.activity_main);

progressBar = findViewById(R.id.progressBar);
btnStart = findViewById(R.id.btnStart);
btnStart.setOnClickListener(v -> {
    progressStatus = 0;
    progressBar.setProgress(progressStatus);
    new Thread(() -> {
        while (progressStatus < 100) {
            progressStatus += 10;
            handler.post(() -> progressBar.setProgress(progressStatus));
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }).start();
});
}

```

activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"

```

```
android:orientation="vertical"
```

```
android:gravity="center"
```

```
android:padding="20dp">
```

```
<ProgressBar
```

```
    android:id="@+id/progressBar"
```

```
    style="?android:attr/progressBarStyleHorizontal"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:max="100"/>
```

```
<Button
```

```
    android:id="@+id/btnStart"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Start Progress"
```

```
    android:layout_marginTop="20dp"/>
```

```
</LinearLayout>
```

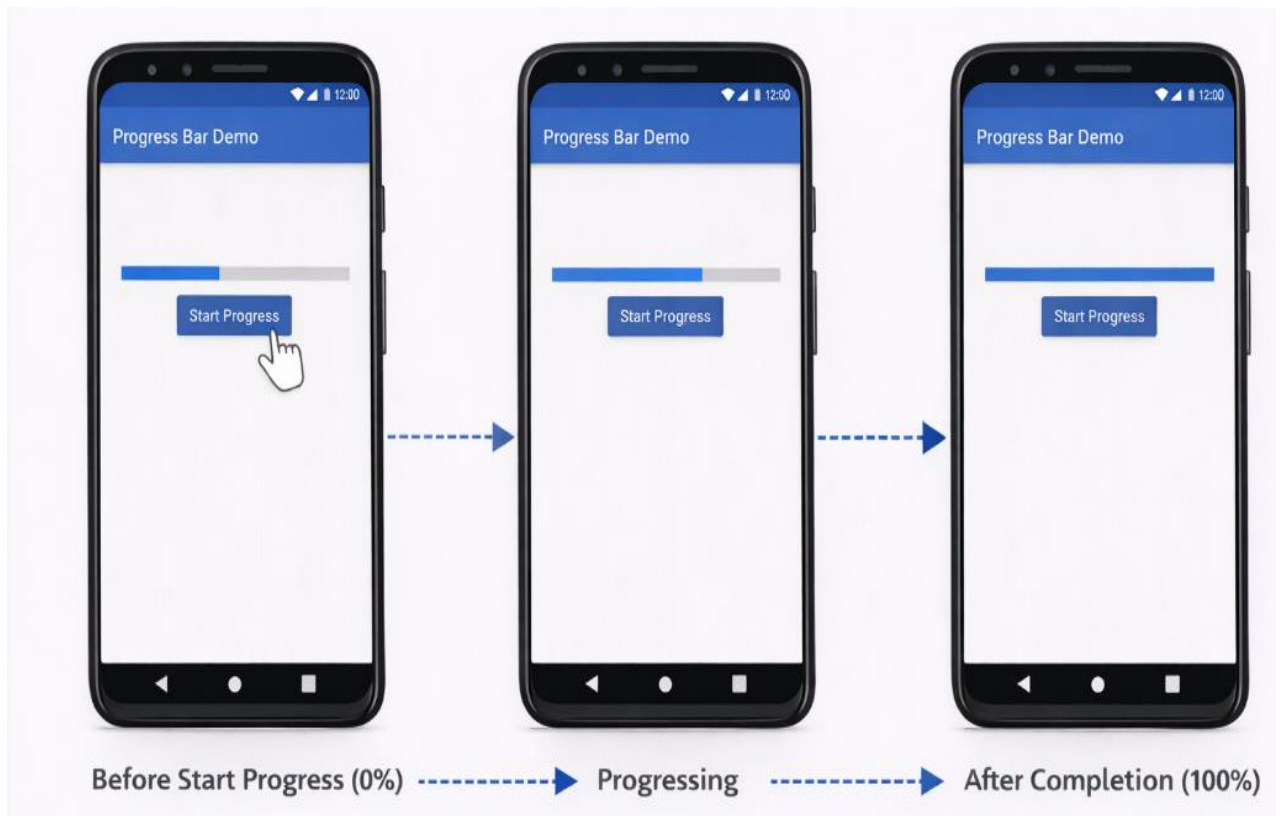
Output

When you click Start Progress button:

The Progress Bar starts from 0%

It increases gradually

Stops automatically at 100%



Conclusion: This practical demonstrates how to use a Progress Bar in an Android application to show task completion status. It helps in understanding how to update UI components dynamically and improves user experience by indicating loading or processing progress.

Submitted By:

Checked By: Ms.Swati.S.Badhe

Sign:

Nam:

Roll No: